

SAFECAST: Robust Failure Detection for VLA Policies with Contrast-Set Training and Calibration

Harshitha Rajaprakash

Aditeya Prajapati

Rong Xue

Abrar Anwar

Jesse Thomason

University of Southern California, Thomas Lord Department of Computer Science
rajaprak@usc.edu

Abstract:

Vision-language-action policies often fail under deployment-time distribution shifts such as clutter, distractor objects, lighting changes, novel objects, altered initial states, and reworded instructions. Hidden-state-based risk probes combined with functional conformal prediction can detect rollout failures, but their reliability depends on calibration data matching deployment conditions. We introduce SAFECAST, which leverages contrast set perturbations to improve hidden-state probe training and calibration for deployment time shift. SAFECAST statistically significantly improves failure detection ROC-AUC scores over a state of the art baseline in both real-world DROID and LIBERO simulation experiments across multiple VLM backbones. We further find that SAFECAST benefits most when both visual and language contrast set perturbations are used to augment data, and that with contrast set perturbations, sim-to-real calibration leads to better probes than using real rollout data only.

Keywords: VLA reliability, Failure detection, Contrast sets

1 Introduction

Vision-language-action (VLA) policies enable robots to perform manipulation tasks from natural language instructions. Models such as OpenVLA [1] and π_0 [2] exhibit promising generalization across tasks, objects, and environments. However, a policy that succeeds in a clean training-like setting may fail when exposed to clutter, distractor objects, lighting variation, novel object appearances, changed robot initial states, or reworded instructions. These failures are concerning in safety-critical settings such as assistive robotics, human-robot interaction, and manipulation of fragile objects.

A growing line of work studies runtime failure prediction for robotic policies. The existing **ScAlable Failure Estimation (SAFE)** [3]-style methods use hidden states of VLA policies to predict failure risk and apply functional conformal prediction to obtain temporally calibrated risk thresholds, flagging unsafe rollouts before task failure occurs. However, their reliability depends on a key assumption: that the calibration trajectories resemble the trajectories encountered during deployment. Deployment rollouts may differ due to visual shifts, language shifts, or their combination. Under such multimodal shifts, the distribution of hidden-state risk trajectories can change substantially, causing thresholds calibrated only on source-distribution rollouts to become unreliable.

This work introduces **SAFECAST (ScAlable Failure Estimation with Contrast-set Augmentation for Safety Tracking)**, a contrast-set-aware framework for robust hidden-state failure detection for VLAs. SAFECAST incorporates visual and language contrast-set trajectories during probe training and conformal calibration (Figure 1). We find that SAFECAST provides more reliable failure detection during distribution shifts than vanilla SAFE, that joint visual-language contrast sets improve robustness more than single-modality contrast sets, and that SAFECAST can be trained in simulation and calibrated on physical rollouts to outperform more expensive real world probe training.

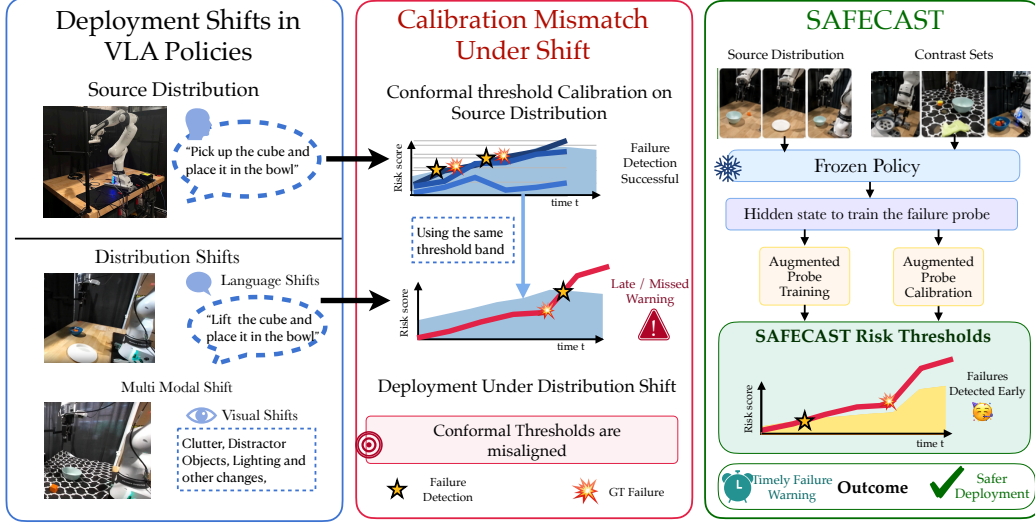


Figure 1: **SAFECAST**. *Left*: VLA policies trained on source-distribution rollouts can encounter visual, language, and multimodal deployment shifts during execution. *Middle*: Hidden-state failure detectors calibrated only on source-distribution trajectories can become misaligned under deployment mismatch, leading to delayed or missed failure warnings. *Right*: SAFECAST intervenes at probe training and conformal calibration time with contrast-set trajectories generated from perturbed in-distribution conditions, producing more reliable time-varying risk thresholds during deployment.

2 Related Works

Vision-language-action (VLA) policies demonstrate strong manipulation capabilities but remain sensitive to deployment-time distribution shifts. Our work builds on runtime failure detection and conformal calibration for robot policies and studies whether contrast-set perturbations can improve failure detection robustness under distribution shifts.

Vision-Language-Action (VLA) Models have demonstrated strong generalization across manipulation tasks by directly mapping visual observations and language instructions to robot actions [4, 5, 1, 6, 7, 8]. These systems leverage large-scale robot demonstration data to enable broader capabilities without task-specific engineering. However, prior work has shown that VLA performance degrades substantially under deployment-time perturbations such as clutter, lighting variation, and language perturbation [9, 10, 11, 12, 13, 14, 15, 16, 17]. While benchmark performance continues to improve through scaling [18, 19, 20, 21], reliable deployment remains challenging due to sensitivity to out-of-distribution conditions. Our work builds on this setting and focuses on improving runtime reliability under deployment shift.

Runtime Failure Detection for Robot Policies is critical for safely deploying learned robot policies in real-world environments, where even small execution errors can lead to unsafe or irreversible outcomes [22, 23, 24]. Prior work has studied failure detection through out-of-distribution monitoring [25, 26, 27, 28, 29, 30] and supervised failure prediction [31, 32, 33, 34, 35, 36]. Several methods also estimate model uncertainty as a proxy for failure. In language models, token-level uncertainty and semantic disagreement across sampled generations have been used to detect hallucinations and unreliable outputs [37, 38, 39], and have been adapted as baselines for VLA failure detection [3]. Other approaches use large vision-language models [40, 41] or reward models [42] for success or failure detection. While these methods provide useful runtime monitoring signals, many require multiple action samples, additional model queries, or expensive VLM inference. We build on state of the art work on multitask failure detection for vision-language-action policies that probes hidden policy representations and calibrating time-varying failure thresholds [3]. Internal policy representations contain information predictive of downstream failures. However, existing failure detectors are

generally calibrated under source-distribution conditions. We directly study how to improve their reliability when deployment conditions differ substantially from calibration data.

Conformal Calibration Under Deployment Shift. Conformal prediction provides calibrated decision thresholds [43, 44], and functional conformal prediction extends these ideas to time-varying signals [45]. Recent robot failure detection methods use conformal prediction to set time-varying thresholds for sequential failure scores [25, 3, 35]. However, these guarantees rely on calibration rollouts being representative of deployment rollouts, an assumption frequently violated in NLP [46] and robotics. Contrast sets have been used to expose model brittleness through targeted perturbations in NLP, vision-language reasoning, and robotics [47, 11]. Our work uses contrast-set perturbations to study and improve calibration robustness, finding that perturbation-aware probe training and calibration can strengthen VLA failure detection under distribution shifts.

3 Background and Task Definiton

Our method builds on **ScAlable Failure Estimation (SAFE)** [3], a hidden-state risk probing framework for runtime failure detection in vision-language-action (VLA) policies. SAFE takes as input the hidden state of a frozen VLA policy at timestep t together with a conformal significance level α , corresponding to different operational risk tolerances during deployment, and predicts whether the ongoing rollout is likely to fail. The probe is trained on VLA rollouts labeled with success or failure to predict, at every timestep, whether failure will occur. The probe’s temporally-dependent threshold for runtime failure detection is calibrated with functional conformal prediction.

Our proposed intervention targets the data distributions used for probe training and conformal calibration. Inspired by contrast sets [11], we augment these datasets with perturbed rollout trajectories constructed through visual and language perturbations that require minimal experimenter overhead. We denote this framework as **SAFECAST (ScAlable Failure Estimation with Contrast-Set Augmentation for Safety Tracking)**.

Vision-Language-Action (VLA) Policies map visual observations and language instructions to robot actions. Let π denote a frozen VLA policy. At timestep t , the policy receives a visual observation o_t and language instruction l , and predicts an action $a_t \sim \pi(\cdot \mid o_t, l)$. Executing π over a horizon of length T produces a rollout $\tau = \{(o_t, l, a_t)\}_{t=1}^T$, where o_t depends on previous actions and environment state while l remains fixed throughout the rollout. Let $\mathcal{D}_\pi^{\text{train}}$ denote the data used to train and finetune the VLA policy π . We fix π after this training; the VLA policy is not updated during failure-probe training, conformal calibration, or evaluation.

Functional Conformal Prediction for VLAs. SAFE [3] probes VLA hidden states to predict whether an ongoing rollout will eventually fail. Given a rollout τ , the frozen VLA policy produces a sequence of hidden states during execution. For each timestep t , let h_t denote the pre-final-layer hidden representation extracted from π . A lightweight MLP probe f_ϕ maps this hidden representation to a scalar failure score, $r_t = f_\phi(h_t)$. The score r_t is interpreted as the estimated risk that the rollout will eventually fail, conditioned on the current hidden state.

For a rollout of length T , applying the probe at every timestep produces a temporally aligned risk score sequence $r_{1:T} = (r_1, r_2, \dots, r_T)$. This risk sequence is used for both conformal calibration and test-time failure detection.

After probe training, functional conformal prediction calibrates the time-varying failure threshold δ_t using calibration rollouts τ_j^{cal} . For each calibration rollout, the trained probe produces a trajectory of risk scores $r_{1:T}^{(j)}$, from which functional conformal prediction computes nonconformity scores, S_{cal} over successful calibration trajectories to construct a one-sided time-varying threshold $\delta_t = \mu_t + q_\alpha$, and $\alpha \in (0, 1)$ controls the deployment-time operational risk tolerance.

Lower α values yield higher risk thresholds leading to lower false positives and higher α results in a lower threshold which results in a higher false positive rate. During deployment, the detector flags a rollout as headed for failure whenever $r_t > \delta_t$.

Problem Definition: Failure Detection Under Distribution Shift. This work studies reliable rollout-level failure detection for frozen VLA policies under deployment-time distribution shift. Let $\mathcal{D}_{\text{src}}$ denote the source-distribution rollout pool from which probe training and calibration data are sampled. Let $\mathcal{D}_{\text{eval}}$ denote the evaluation distribution encountered during deployment. The evaluation distribution may differ from $\mathcal{D}_{\text{src}}$ due to visual, language or multimodal their combination.

4 SAFECAST

We introduce SAFECAST for robust failure detection using hidden-state probes under multimodal deployment shift using contrast set training and calibration. Given VLA rollouts from the source distribution $\mathcal{D}_{\text{src}}$ and a contrast-set pool $\mathcal{D}_{\text{CS}}$, hidden states are extracted from the frozen VLA policy, a lightweight risk probe is trained to predict timestep-wise failure scores, and functional conformal prediction is used to calibrate time-varying failure thresholds. The key question is whether SAFECAST training and calibration improve failure detection on the evaluation distribution $\mathcal{D}_{\text{eval}}$.

Contrast Sets for Failure Probing. Recall that $\mathcal{D}_{\text{src}}$ denotes the source rollout distribution from which the probe training set $\mathcal{D}_{\phi}^{\text{train}}$ and calibration set $\mathcal{D}_{\phi}^{\text{cal}}$ are drawn. A source-distribution rollout is denoted as $\tau^{\text{src}} = \{(o_t, l, a_t)\}_{t=1}^T$, where o_t is the visual observation, l is the language instruction, and a_t is the action produced by the frozen VLA policy.

SAFECAST constructs a contrast-set rollout pool $\mathcal{D}_{\text{CS}}$ by applying controlled perturbation operators to the visual and/or language inputs of source-distribution rollouts and then re-executing the frozen VLA policy under the perturbed condition. A contrast-set rollout is denoted as $\tau^{\text{CS}} = \{(\tilde{o}_t, \tilde{l}, \tilde{a}_t)\}_{t=1}^T$, where $\tilde{o}_t$ is a visually perturbed observation, $\tilde{l}$ is a perturbed or paraphrased language instruction, and $\tilde{a}_t$ is the action produced by re-executing the policy with these perturbed inputs. Because the policy is re-executed, τ^{CS} is not treated as a relabeled or counterfactual copy of τ^{src} ; the resulting action sequence $\tilde{a}_{1:T}$ and hidden-state trajectory may differ from the source rollout. Thus, we do not assume trajectory-level exchangeability between $\mathcal{D}_{\text{src}}$ and $\mathcal{D}_{\text{CS}}$. Instead, contrast sets are used to construct training and calibration distributions whose risk trajectories better approximate deployment shifts expected in $\mathcal{D}_{\text{eval}}$.

We use this family of contrast-set perturbations, $\mathcal{D}_{\text{CS}} = \mathcal{D}_{\text{CS}}^{\text{vis}} \cup \mathcal{D}_{\text{CS}}^{\text{lang}}$, where $\mathcal{D}_{\text{CS}}^{\text{vis}}$ contains visual contrast-set rollouts, and $\mathcal{D}_{\text{CS}}^{\text{lang}}$ contains language contrast-set rollouts. The contrast-set-augmented rollout pool is $\mathcal{D}_{\text{aug}} = \mathcal{D}_{\text{src}} \cup \mathcal{D}_{\text{CS}}$. Depending on the configuration, probe training and/or conformal calibration draw data from $\mathcal{D}_{\text{aug}}$ rather than only from $\mathcal{D}_{\text{src}}$.

Visual contrast sets. Visual contrast sets perturb the observation stream while keeping the language instruction fixed. In LIBERO, we construct visual contrast sets by adding distractor objects to LIBERO-Spatial scenes. Distractors are selected so that they do not change the intended task, are not referenced in the language instruction, and do not obstruct the nominal source-distribution rollout. Thus, the target object, action, and goal remain unchanged, while the visual context differs from the source distribution. In real-world DROID experiments, visual perturbations include distractor objects, cluttered backgrounds, novel objects, and altered object configurations. Additionally, DROID visual perturbations result in novel rollouts. For more details on the number of rollouts used per perturbation, please refer to Appendix A

Language contrast sets. Language contrast sets perturb the instruction while keeping the visual scene fixed. In LIBERO, we generate three paraphrases for each LIBERO-Spatial task, by prompting the ChatGPT 5.1 Instant model. For more details regarding the paraphrases and prompt used to generate them, refer to Appendix A. These are designed to preserve the intended target object, action, and goal location. To reduce rollout collection cost, each paraphrase is evaluated on three episodes per task rather than all 50 source episodes. These perturbations test whether the failure

Table 1: Failure detection performance across simulation and real-world settings comparing SAFE and SAFECAST variants. Contrast set-aware probe training and calibration consistently improve failure detection over the SAFE method across both F1 and ROC-AUC. F1 measures the balance between precision and recall for rollout-level failure detection, while ROC-AUC measures how well the failure scores separate successful and failed rollouts. Averaged over $\alpha \in \{0.1, 0.2, \dots, 0.9\}$ and 30 seeds). **Bold** indicates maximum. * indicates statistical significance, tested for ROC-AUC across paired performance at each α threshold. Exact p values are reported Appendix C.

Method	F1				ROC-AUC			
	Sim- π_0	Sim-OpenVLA	Real- π_0	Real- π_0 -FAST	Sim- π_0	Sim-OpenVLA	Real- π_0	Real- π_0 -FAST
<i>Always guess failure</i>	0.5266	0.8314	0.7322	0.7654	0.0000	0.0000	0.0000	0.0000
SAFE	0.7132	0.8589	0.4085	0.7475	0.3267	0.5937	0.2626	0.5501
SAFECAST _{TRAINAUG}	0.7516	0.8687	0.5838	0.8109	0.4278*	0.6514	0.2649	0.6621*
SAFECAST _{CAL AUG}	0.7373	0.8628	0.4480	0.7534	0.4382*	0.6782*	0.3415*	0.6578*
SAFECAST	0.7528	0.8728	0.5412	0.8053	0.4469*	0.8014	0.3807*	0.6664*

detector remains reliable when task semantics are preserved but the linguistic form changes. In real-world DROID experiments, language perturbations include paraphrased instructions and negated or distractor phrasing when applicable. These were generated by giving a prompt to ChatGPT 5.1.

Joint visual-language contrast sets. Joint visual-language contrast sets perturb both the observation stream and the language instruction. These rollouts combine visual perturbations with paraphrased instructions and therefore induce multimodal deployment shifts. They are intended to approximate deployment conditions that may violate the exchangeability assumptions underlying source-distribution conformal calibration. Examples of these are primarily in the real setting.

Candidate training rollouts are filtered using an active rejection procedure based on Dynamic Time Warping (DTW). Candidates whose trajectories are too similar to already-selected rollouts are rejected using a fixed DTW distance threshold, reducing near-duplicates in $\mathcal{D}_{\phi, \text{CS}}^{\text{train}}$. For more details, refer to B.3 The augmented probe-training set is then $\mathcal{D}_{\phi, \text{aug}}^{\text{train}}$. For conformal calibration, the contrast-set calibration pool is $\mathcal{D}_{\phi, \text{CS}}^{\text{cal}}$ and the augmented calibration set is $\mathcal{D}_{\phi, \text{aug}}^{\text{cal}}$. Calibration subsets are sampled with controlled success and failure counts where possible so that threshold estimation is not dominated by a single outcome class. Appendix E.1 reports the number of rollouts collected for each perturbation type and split.

Contrast Set-Augmented Probe Training and Calibration. SAFE probe training and calibration sample rollouts from $\mathcal{D}_{\phi, \text{src}}^{\text{train}}$ and $\mathcal{D}_{\phi, \text{src}}^{\text{cal}}$, respectively, while SAFECAST instead uses the augmented rollout datasets $\mathcal{D}_{\phi, \text{aug}}^{\text{train}}$ and $\mathcal{D}_{\phi, \text{aug}}^{\text{cal}}$ containing contrast-set trajectories.

5 Experiments and Results

The main findings of our experiments are that (1) SAFECAST provides more reliable failure detection during distribution shifts than vanilla SAFE; (2) joint visual-language contrast sets improve robustness more than single-modality contrast sets; and (3) SAFECAST can be trained in simulation and calibrated on physical rollouts to outperform more expensive real world probe training.

5.1 Experimental Setup

We conduct experiments comparing SAFE and SAFECAST on both a physical robot and in a simulation environment. To isolate the roles of contrast-set probe training and contrast-set conformal calibration proposed as part of SAFECAST, we evaluate four configurations (Table 1). Additional details on the specific conditions and examples of DROID tasks, and on LIBERO perturbation procedures, policy performance, dataset filtering, and the simulation setup are included in Appendix B.

We evaluate SAFECAST on a Franka/DROID setup using π_0 and π_0 -FAST. For π_0 , the open-source DROID checkpoint is LoRA-finetuned on $\mathcal{D}_{\pi}^{\text{train, real}}$ consisting of 20 teleoperated spatial

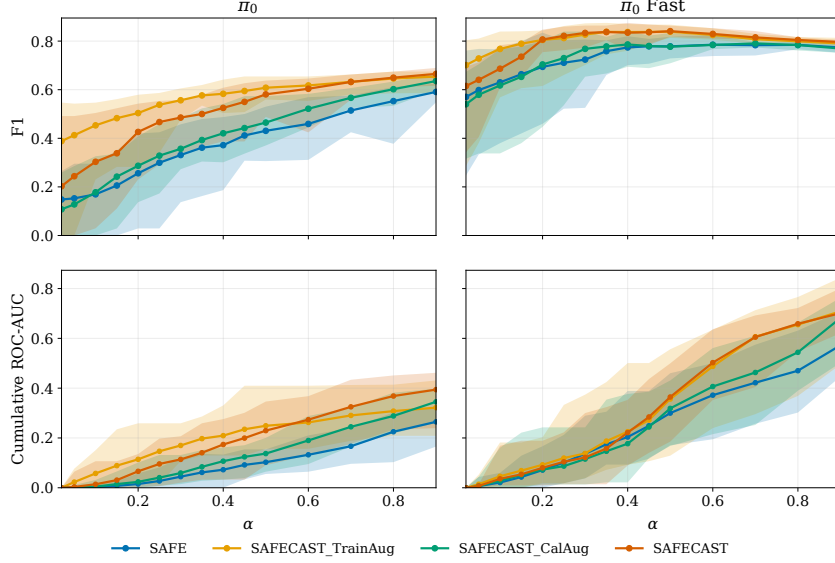


Figure 2: Real-world failure detection across varying α on the DROID setup. Top: F1-score across calibrated intervention thresholds. Bottom: ROC-AUC measuring separation between successful and failed rollouts. Lower α values correspond to more conservative deployment regimes with stricter intervention thresholds. Across both π_0 and π_0 -FAST, SAFECAST probes consistently improve failure detection performance over SAFE, with the strongest gains appearing when both augmented training and calibration are combined in SAFECAST. Averaged over 30 seeds.

pick-and-place demonstrations, as the zero-shot checkpoint achieved zero successful rollouts in preliminary testing. The π_0 -FAST checkpoint is evaluated without additional finetuning since it achieved nonzero task success in the same setup. Evaluation is performed on deployment rollouts $\tau^{\text{eval}} \in \mathcal{D}_{\text{eval}}^{\text{real}}$ containing stronger shifts including novel objects, new tasks, clutter, lighting variation, and altered robot initial states.

We evaluate OpenVLA and π_0 on LIBERO-Spatial and LIBERO-Plus [20]. Unlike SAFE [3], which splits LIBERO-Spatial into seen and unseen tasks from the same distribution, we separate source, calibration, and deployment distributions to study robustness under controlled deployment shift.

Metrics. Following prior work in uncertainty estimation and VLA failure detection [38, 37, 3], we evaluate failure detectors using F1-score and ROC-AUC across sweeps of the conformal significance level α . Each value of α induces a different calibrated intervention threshold, corresponding to a different deployment-time risk tolerance. Lower α values result in lower false positive rates (FPR) of flagging failure, while higher α values yield higher FPR, with more trajectories being flagged as headed for failure. We report α -marginalized F1 by averaging over the evaluated α sweep, capturing robustness across risk tolerances rather than performance at a single selected threshold. We compute TPR and FPR at each α and report ROC-AUC as the area under the resulting TPR-FPR curve.

5.2 SAFECAST improves failure detection during distribution shifts.

In the real world DROID experiments, SAFECAST substantially improves deployment robustness for both π_0 and π_0 -FAST (Table 1) across a broad range of risk tolerances (Figure 2). Interestingly, the relative benefits of augmented probe training and augmented calibration differ across policies. For π_0 , augmented probe training provides the largest gains, suggesting that exposure to perturbed rollout trajectories during representation learning substantially improves downstream failure separability. In contrast, π_0 -FAST benefits more strongly from SAFECAST calibration, possibly indicating that calibration mismatch between source-distribution trajectories and deployment conditions

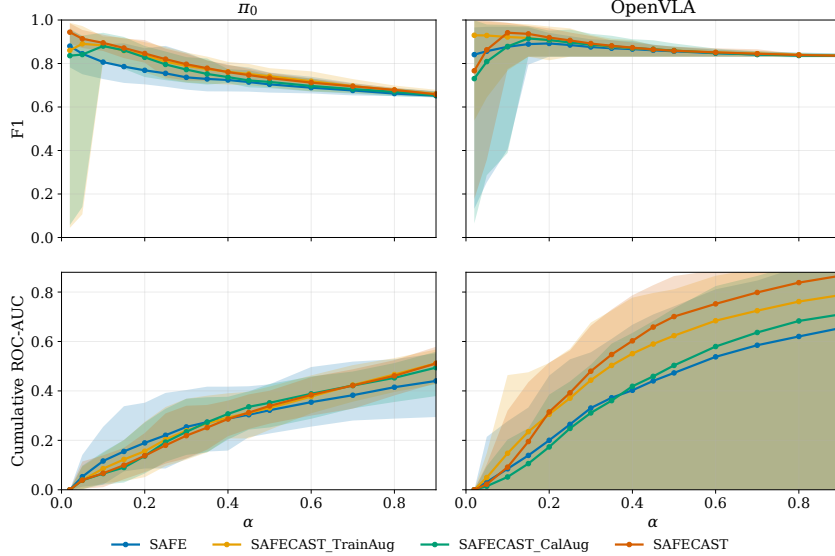


Figure 3: Simulation failure detection across varying α on LIBERO. Top: F1 across the α values. Bottom: cumulative ROC-AUC across the α values. Across both π_0 and OpenVLA, contrast-set-aware probes consistently improve failure detection performance over SAFE, with SAFECAST and augmented probe training producing the strongest overall improvements.

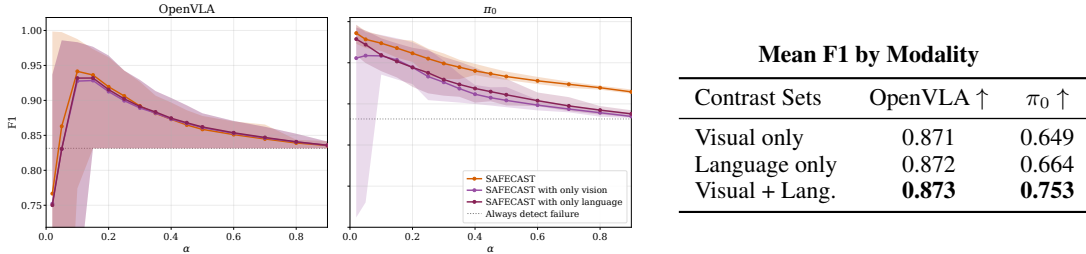


Figure 4: Effect of contrast-set modality on simulation failure detection. Left: F1 across conformal significance levels α for OpenVLA and π_0 . Right: peak F1 for each contrast-set modality. Joint visual-language contrast sets perform best for both policies.

becomes a larger bottleneck for stronger pretrained policies. Across both policies, SAFECAST either achieves the best overall performance or closely matches the strongest individual augmentation strategy and suggests that jointly incorporating perturbation-aware probe training and calibration produces the most consistently robust deployment-time failure detection behavior.

We find these results hold similarly in simulation. In LIBERO, we show a similar trend in Table 1 and Figure 3. Across both OpenVLA and π_0 , contrast-set-aware configurations consistently improve failure detection robustness over SAFE. As in the real-world setting, the strongest augmentation strategy varies across policies: OpenVLA benefits most from augmented probe training, while π_0 achieves the strongest overall robustness under full SAFECAST training and calibration. Together, these results support that SAFECAST strengthens the reliability of failure detection during diverse deployment conditions across various risk tolerances.

5.3 Visual + Language Contrast Sets Improve Failure Detection Robustness

Figure 4 ablates visual-only, language-only, and joint visual-language contrast sets under the best-performing SAFECAST configuration from the simulation results in Section 5.2. Across both OpenVLA and π_0 , jointly incorporating visual and language perturbations produces the strongest OOD failure detection robustness. For example, on OpenVLA, multimodal contrast sets improve F1 from

0.922 under visual-only augmentation to 0.959. While both visual-only and language-only contrast sets improve robustness, the best performance is consistently achieved when both modalities are included together, suggesting that deployment-time calibration benefits from covering the joint perturbation space expected during execution rather than modeling visual or language shifts independently. Please refer to E.2 for ROC-AUC analysis.

5.4 SAFECAST Probe Training in Simulation Calibrates DROID Failure Detection

Since it is easier to collect large amounts of data in simulation, we train failure probes on trajectories generated in LIBERO-Spatial and then adapt calibration using a smaller real-world contrast-set dataset. Specifically, the probe is trained entirely in simulation while functional conformal calibration is performed using real-world contrast-set trajectories collected on the DROID setup. This sim-training to real-contrast-set-calibration improves failure detection robustness compared to only training and calibrating a probe only on the smaller set of real-world data (Figure 5). For additional analyses of ROC-AUC, please refer to E.3

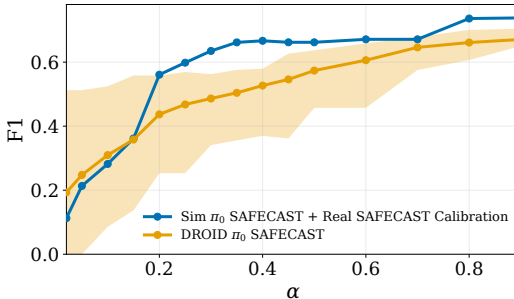


Figure 5: Sim-to-real failure detection robustness across varying α . SAFECAST enables sim-to-real transfer even under deployment mismatch.

6 Conclusion

We introduced SAFECAST, a framework for improving hidden-state failure detection in vision-language-action policies through contrast-set-aware probe training and conformal calibration. SAFECAST addresses a key limitation of source-distribution failure calibration: deployment roll-outs may induce visual, language, multimodal, or sim-to-real shifts that change the hidden-state risk trajectories used for failure detection. By incorporating contrast-set trajectories during probe training and/or calibration, SAFECAST improves alignment between calibration and deployment risk distributions.

Across LIBERO simulation, real-world DROID/Franka experiments, and sim-to-real transfer, SAFECAST improves runtime failure detection under deployment shift. The results show that contrast-set-aware configurations improve both F1 and ROC-AUC across conformal significance levels α . This supports the central claim that robust failure detection should be evaluated not only at a single operating point, but across deployment risk tolerances.

Limitations. SAFECAST depends on collecting contrast-set trajectories that are representative of likely deployment shifts. If the robot encounters perturbations outside the contrast-set distribution, failure detection performance may still degrade. Additionally, contrast-set collection introduces extra rollout cost, and our current perturbation families cover only a subset of possible visual, language, and task-level changes. The method also improves calibration alignment empirically, but does not restore formal conformal exchangeability guarantees under arbitrary deployment shift.

Future work could study adaptive contrast-set construction, where new perturbations are generated online based on detector uncertainty or observed failure modes. Another promising direction is integrating SAFECAST with human-in-the-loop learning. Calibrated risk estimates could trigger human intervention during execution, allowing supervisors to stop unsafe rollouts, provide corrective actions, or teleoperate through difficult states. These interventions could then be incorporated back into the policy training and failure-calibration datasets, moving from passive failure detection toward interactive policy improvement.

Acknowledgments

This work was funded in part by a DARPA ARC Safe and Assured Foundation Robots for Open Environments (SAFRON) grant (Award HR0011-25-3-0154). The authors would like to thank grant Co-PIs Souti Chattopadhyay and William G.J. Halfond for brainstorming discussions.

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti. Safe: Multitask failure detection for vision-language-action models. In *Neural Information Processing Systems (NeurIPS)*, 2025.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023.
- [5] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. *Conference on Robot Learning (CoRL)*, 2023.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control. *CoRR*, abs/2410.24164, 2024.
- [7] Physical Intelligence, K. Black, N. Brown, D. Driess, C. Finn, S. Levine, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [8] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [9] W. Pumacay, I. Singh, J. Duan, R. Krishna, J. Thomason, and D. Fox. THE COLOSSEUM: A Benchmark for Evaluating Generalization for Robotic Manipulation. *Robotics: Science and Systems (RSS)*, 2024.
- [10] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- [11] A. Anwar, R. Gupta, and J. Thomason. Contrast sets for evaluating language-guided robot policies. *Conference on Robot Learning (CoRL)*, 2024.

- [12] A. Anwar, R. Gupta, Z. Merchant, S. Ghosh, W. Neiswanger, and J. Thomason. Efficient evaluation of multi-task robot policies with active experiment selection. In *Conference on Robot Learning (CoRL)*, 2025.
- [13] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy, P. Shah, P. Horgan, G. Richardson, S. Feng, and B. Burchfiel. Robot learning as an empirical science: Best practices for policy evaluation. *arXiv*, 2024.
- [14] A. Parekh, N. Vitsakis, A. Suglia, and I. Konstas. Investigating the Role of Instruction Variety and Task Difficulty in Robotic Manipulation Tasks. *arXiv preprint arXiv:2407.03967*, 2024.
- [15] A. Xie, L. Lee, T. Xiao, and C. Finn. Decomposing the generalization gap in imitation learning for visual robotic manipulation. *International Conference on Robotics and Automation (ICRA)*, 2024.
- [16] J. Gao, S. Belkhale, S. Dasari, A. Balakrishna, D. Shah, and D. Sadigh. A taxonomy for evaluating generalist robot policies. *RA-L*, 2025.
- [17] C. Mitra, Y. Luo, R. Saravanan, D. Niu, A. Pai, J. Thomason, T. Darrell, A. Anwar, D. Ramanan, and R. Herzig. Mechanistic finetuning of vision-language-action models via few-shot demonstrations. *arXiv preprint*, 2025.
- [18] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. *arXiv preprint arXiv:2406.02523*, 2024.
- [19] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *NeurIPS*, 2023.
- [20] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [21] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2019.
- [22] R. Sinha, A. Sharma, S. Banerjee, T. Lew, R. Luo, S. M. Richards, Y. Sun, E. Schmerling, and M. Pavone. A system-level view on out-of-distribution data in robotics. *arXiv preprint arXiv:2212.14020*, 2022.
- [23] R. Natarajan, S. Reddy, S. C. Bose, H. L. Gururaj, F. Flammini, and S. Velmurugan. Fault detection and state estimation in robotic automatic control using machine learning. *Array*, 19: 100298, 2023.
- [24] Q. M. Rahman, P. Corke, and F. Dayoub. Run-time monitoring of machine learning for robotic perception: A survey of emerging trends. *IEEE Access*, 9:20067–20075, 2021.
- [25] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. *arXiv preprint arXiv:2503.08558*, 2025.
- [26] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu. Multi-task interactive robot fleet learning with visual world models. In *Conference on Robot Learning*, 2024.
- [27] R. Sinha, A. Elhafi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone. Real-time anomaly detection and reactive planning with large language models. In *Robotics: Science and Systems (RSS)*, 2024.

- [28] J. Wong, A. Tung, A. Kurenkov, A. Mandlekar, L. Fei-Fei, S. Savarese, and R. Martín-Martín. Error-aware imitation learning from teleoperation data for mobile manipulation. In *Conference on Robot Learning*, 2022.
- [29] A. Majumdar, M. Sharma, D. Kalashnikov, S. Singh, P. Sermanet, and V. Sindhwani. Predictive red teaming: Breaking policies without breaking robots. *arXiv preprint arXiv:2502.06575*, 2025.
- [30] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, pages 689–723. PMLR, 2025.
- [31] H. Liu, S. Dass, R. Martín-Martín, and Y. Zhu. Model-based runtime monitoring with interactive imitation learning. In *IEEE International Conference on Robotics and Automation*, pages 4154–4161. IEEE, 2024.
- [32] C. Gokmen, D. Ho, and M. Khansari. Asking for help: Failure prediction in behavioral cloning through value approximation. In *International Conference on Robotics and Automation (ICRA)*, 2023.
- [33] A. Xie, F. Tajwar, A. Sharma, and C. Finn. When to ask for help: Proactive interventions in autonomous reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2022.
- [34] R. Sinha, E. Schmerling, and M. Pavone. Closing the loop on runtime monitors with fallback-safe mpc. In *Conference on Decision and Control*, pages 6533–6540. IEEE, 2023.
- [35] A. Farid, D. Snyder, A. Z. Ren, and A. Majumdar. Failure prediction with statistical guarantees for vision-based robot control. In *Robotics: Science and Systems*, 2022.
- [36] T. Ablett, F. Maric, and J. Kelly. Fighting failures with fire: Failure identification to reduce expert burden in intervention-based learning. *arXiv preprint arXiv:2007.00245*, 2020.
- [37] Y. Huang, J. Song, Z. Wang, S. Zhao, H. Chen, F. Juefei-Xu, and L. Ma. Look before you leap: An exploratory study of uncertainty measurement for large language models. *arXiv preprint arXiv:2307.10236*, 2023.
- [38] L. Kuhn, Y. Gal, and S. Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations*, 2023.
- [39] O. Shorinwa, Z. Mei, J. Lidard, A. Z. Ren, and A. Majumdar. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *arXiv preprint arXiv:2412.05563*, 2024.
- [40] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.
- [41] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. de Freitas, and S. Cabi. Vision-language models as success detectors. In *Conference on Lifelong Learning Agents*, pages 120–136. PMLR, 2023.
- [42] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *Robotics: Science and Systems (RSS)*, 2026.
- [43] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

- [44] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [45] J. Diquigiovanni, M. Fontana, and S. Vantini. The importance of being a band: Finite-sample exact distribution-free prediction sets for functional data. *Statistica Sinica*, 34:1–41, 2024.
- [46] S. Devic, T. Srinivasan, J. Thomason, W. Neiswanger, and V. Sharan. From calibration to collaboration: Llm uncertainty quantification should be more human-centered. *arXiv preprint arXiv:2506.07461*, 2025.
- [47] M. Gardner, Y. Artzi, V. Basmov, J. Berant, B. Bogin, S. Chen, P. Dasigi, D. Dua, Y. Elazar, A. Gottumukkala, N. Gupta, H. Hajishirzi, G. Ilharco, D. Khashabi, K. Lin, J. Liu, N. F. Liu, P. Mulcaire, Q. Ning, S. Singh, N. A. Smith, S. Subramanian, R. Tsarfaty, E. Wallace, A. Zhang, and B. Zhou. Evaluating models’ local decision boundaries via contrast sets. *Findings of Empirical Methods in Natural Language Processing (EMNLP Findings)*, 2020.

A Contrast-Set Construction

This appendix provides additional details on the construction of source-distribution and contrast-set rollout pools used for SAFECAST training and calibration. All contrast-set rollouts are generated by perturbing the visual and/or language inputs of source-distribution conditions and re-executing the frozen VLA policy under the perturbed condition.

A.1 Rollout Split Statistics

Table 2: Probe training and calibration rollout statistics for source-distribution and contrast-set datasets used in SAFECAST evaluation. Each entry reports the number of labeled rollout pairs (τ, y) in the source-distribution split and the contrast-set split. Source denotes $\mathcal{D}_{\text{src}}$, and CS denotes $\mathcal{D}_{\text{CS}}$.

Setting	Policy	Split	$\mathcal{D}_{\text{src}}$ Succ.	$\mathcal{D}_{\text{src}}$ Fail.	$\mathcal{D}_{\text{CS}}$ Succ.	$\mathcal{D}_{\text{CS}}$ Fail.
$\mathcal{D}_{\text{LIBERO-Spatial}}$	π_0	Train	436	16	72	15
$\mathcal{D}_{\text{LIBERO-Spatial}}$	π_0	Cal.	45	5	53	37
$\mathcal{D}_{\text{LIBERO-Spatial}}$	OpenVLA	Train	369	81	144	112
$\mathcal{D}_{\text{LIBERO-Spatial}}$	OpenVLA	Cal.	42	8	23	94
$\mathcal{D}_{\text{DROID}}$	π_0	Train	17	27	6	26
$\mathcal{D}_{\text{DROID}}$	π_0	Cal.	14	28	9	21
$\mathcal{D}_{\text{DROID}}$	π_0 -FAST	Train	25	22	13	18
$\mathcal{D}_{\text{DROID}}$	π_0 -FAST	Cal.	19	19	12	20

A.2 Perturbation Types

Table 3: Contrast-set perturbations used to construct deployment-shift training and calibration rollouts. Counts report the number of contrast-set rollouts available for each policy and split.

Setting	Perturbation Type	OpenVLA Train	OpenVLA Cal.	π_0 Train	π_0 Cal.
Simulation	Visual distractors	133	107	87	45
Simulation	Language paraphrases	133	25	103	45
Real robot	Visual distractors/clutter	15	8	15	8
Real robot	Language paraphrases/negations	31	30	31	30
Real robot	Visual + language	32	30	32	30
Real robot	Novel objects/tasks	2	8	2	8

Simulation visual perturbations. In LIBERO-Spatial, visual contrast sets are generated by adding distractor objects to the scene. Distractors are selected so that they do not change the intended task, are not referenced in the language instruction, and do not obstruct the nominal source-distribution robot trajectory. Thus, the target object, action, and goal remain unchanged, while the visual context differs from the source distribution.

Simulation language perturbations. For each LIBERO-Spatial task, we generate three semantically equivalent instruction variants that preserve the intended target object, action, and goal location. To reduce rollout collection cost, each paraphrase is evaluated on three episodes per task rather than all 50 source episodes.

Language paraphrase prompt. The paraphrases are generated with the following prompt template:

Rewrite the following robot manipulation instruction in three semantically equivalent ways. Preserve the target object, action, and goal location. Do not introduce new objects, new goals, or ambiguous references. Instruction: “[ORIGINAL INSTRUCTION]”

Example paraphrases. Table 4 shows representative LIBERO-Spatial language contrast-set paraphrases. Each paraphrase preserves the target object, action, and goal location while changing the surface form of the instruction.

Table 4: Representative language contrast-set paraphrases for LIBERO-Spatial tasks.

Paraphrase 1	Paraphrase 2	Paraphrase 3
Lift the black bowl that is placed between the plate and the ramekin, and set it on the plate.	Pick up the black bowl located between the plate and the ramekin, then move it onto the plate.	Grab the black bowl sitting between the ramekin and the plate and place it onto the plate.
Pick up the black bowl from the center of the table and place it on the plate.	Lift the black bowl located at the table center and set it onto the plate.	Grab the black bowl from the middle of the table and put it on the plate.
Lift the black bowl from the cookie box and place it on the plate.	Pick up the black bowl sitting on top of the cookie box and move it to the plate.	Grab the black bowl resting on the cookie box and set it on the plate.
Pick up the black bowl from the top drawer of the wooden cabinet and place it on the plate.	Lift the black bowl inside the upper drawer of the wooden cabinet and move it onto the plate.	Retrieve the black bowl from the wooden cabinet top drawer and put it on the plate.
Lift the black bowl on the ramekin and place it on the plate.	Pick up the black bowl resting on the ramekin and move it onto the plate.	Grab the black bowl sitting on top of the ramekin and set it onto the plate.

Real-world perturbations. In the DROID/Franka setup, visual contrast sets include distractor objects, cluttered backgrounds, novel objects, and altered object configurations. Language contrast sets include paraphrased instructions and negated or distractor phrasing when applicable. Joint visual-language contrast sets combine these visual and language changes, inducing multimodal deployment shifts. Table 5 shows representative real-world source and contrast-set conditions.

Table 5: Examples of real-world DROID source and contrast-set conditions.

Source Task	Language Perturbation	Visual Perturbation
pick up the cube and place it in the bowl	grab the cube and drop it inside the bowl lift the cube and put it into the bowl	Distractor: Plushie Distractor: Marker Distractor: Marker and Plushie Black background texture White background texture
pick up the marker and place it in the bowl	grab the marker and place it inside the bowl pick up the marker and set it in the bowl take the marker near the cup and place it in the bowl	Distractor: Additional object near target Background texture variation Additional object near cup

B Experiment Setup Details

This section provides additional setup details for the simulation and real-world experiments, including perturbation procedures, dataset filtering, and task conditions.

B.1 LIBERO Simulation Setup

In simulation, source-distribution rollouts are collected from LIBERO-Spatial. Evaluation is performed on LIBERO-Plus, which contains deployment-time shifts relative to the source distribution. Contrast-set rollouts are generated by perturbing LIBERO-Spatial source conditions through visual distractors or language paraphrases, then re-executing the frozen VLA policy.

B.2 DROID Real-World Setup

In the real-world setup, π_0 and π_0 -FAST are evaluated on spatial pick-and-place tasks using a Franka/DROID setup. The source-distribution rollouts use fixed task families, while contrast-set and evaluation rollouts introduce visual, language, and joint visual-language shifts.

Algorithm 1 DTW-based active rejection for contrast-set rollout collection

Require: Task t , outcome budgets K_{succ} and K_{fail} , threshold $\tau_{\text{DTW}} = 0.02$

```
1: Initialize accepted pools  $\mathcal{A}_{t,\text{succ}} \leftarrow \emptyset, \mathcal{A}_{t,\text{fail}} \leftarrow \emptyset$ 
2: while  $|\mathcal{A}_{t,\text{succ}}| < K_{\text{succ}}$  or  $|\mathcal{A}_{t,\text{fail}}| < K_{\text{fail}}$  do
3:   Execute policy under the perturbed contrast-set condition to obtain rollout  $\tau$  and label  $y \in \{\text{succ}, \text{fail}\}$ 
4:   Form end-effector trajectory  $\tilde{\tau} \leftarrow \text{NormalizeEE}(\tau)$  by subtracting the initial position
5:   if  $|\mathcal{A}_{t,y}| < K_y$  then
6:     if  $\mathcal{A}_{t,y} = \emptyset$  or  $\min_{\tilde{\tau}_j \in \mathcal{A}_{t,y}} d_{\text{DTW}}(\tilde{\tau}, \tilde{\tau}_j) \geq \tau_{\text{DTW}}$  then
7:       Accept  $\tau$  and set  $\mathcal{A}_{t,y} \leftarrow \mathcal{A}_{t,y} \cup \{\tilde{\tau}\}$ 
8:     else
9:       Reject  $\tau$ 
10:    end if
11:  else
12:    Reject  $\tau$ 
13:  end if
14: end while
```

B.3 Dynamic Time Warping Filtering

For probe training, candidate contrast-set rollouts are filtered using an active rejection procedure based on Dynamic Time Warping (DTW). The goal of this filtering step is to reduce near-duplicate trajectories in the contrast-set training pool and increase diversity among selected perturbation rollouts. Given a candidate rollout, we compute its DTW distance to already-selected contrast-set rollouts from the same task and outcome class. If the candidate is too similar to an existing selected rollout under a fixed DTW distance threshold, it is rejected; otherwise, it is added to the training pool.

Trajectory representation. Each rollout is represented by the time series of 3D robot end-effector positions at every control step. Before DTW, each sequence is translation-normalized by subtracting its initial end-effector position.

DTW distance and threshold. We compute FastDTW with Euclidean distance between 3D end-effector waypoints and divide the resulting distance by the optimal warping path length. A candidate is accepted only if its minimum normalized DTW distance to accepted rollouts in the same task-and-outcome pool is at least

$$\tau_{\text{DTW}} = 0.02.$$

We use $\tau_{\text{DTW}} = 0.02$ during LIBERO contrast-set rollout collection for both OpenVLA and π_0 .

Active-rejection procedure. Algorithm 1 summarizes the active rejection procedure. Success and failure pools are maintained separately for each task; rejected rollouts are not saved to $\mathcal{D}_{\phi,\text{CS}}^{\text{train}}$. This produces the contrast-set training pool

$$\mathcal{D}_{\phi,\text{CS}}^{\text{train}} = \mathcal{D}_{\phi,\text{vis}}^{\text{train}} \cup \mathcal{D}_{\phi,\text{lang}}^{\text{train}} \cup \mathcal{D}_{\phi,\text{vis+lang}}^{\text{train}},$$

and the augmented probe-training set

$$\mathcal{D}_{\phi,\text{aug}}^{\text{train}} = \mathcal{D}_{\phi,\text{src}}^{\text{train}} \cup \mathcal{D}_{\phi,\text{CS}}^{\text{train}}.$$

For conformal calibration, the contrast-set calibration pool is

$$\mathcal{D}_{\phi,\text{CS}}^{\text{cal}} = \mathcal{D}_{\phi,\text{vis}}^{\text{cal}} \cup \mathcal{D}_{\phi,\text{lang}}^{\text{cal}} \cup \mathcal{D}_{\phi,\text{vis+lang}}^{\text{cal}},$$

and the augmented calibration set is

$$\mathcal{D}_{\phi,\text{aug}}^{\text{cal}} = \mathcal{D}_{\phi,\text{src}}^{\text{cal}} \cup \mathcal{D}_{\phi,\text{CS}}^{\text{cal}}.$$

Calibration subsets are sampled with controlled success and failure counts where possible so that threshold estimation is not dominated by a single outcome class.

Table 6: Seed-paired t -tests for full-curve functional CP ROC-AUC over $\alpha \in \{0.1, \dots, 0.9\}$. Each cell reports mean difference (other – SAFE) and raw p -value. * marks Bonferroni significance over $m = 12$ tests.

Comparison	Sim- π_0	Sim-OpenVLA	Real- π_0	Real- π_0 -FAST
SAFECAST _{TRAINAUG} vs. SAFE	+0.1011* ($p < 10^{-10}$)	+0.0647 ($p = 0.4093$)	+0.0023 ($p = 0.8753$)	+0.1120* ($p < 10^{-5}$)
SAFECAST _{CALAUG} vs. SAFE	+0.1115* ($p < 10^{-10}$)	+0.0773* ($p = 0.0008$)	+0.0789* ($p < 10^{-10}$)	+0.1077* ($p < 10^{-7}$)
SAFECAST vs. SAFE	+0.1202* ($p < 10^{-10}$)	+0.2023 ($p = 0.0256$)	+0.1181* ($p < 10^{-8}$)	+0.1162* ($p < 10^{-7}$)

C Statistical Testing Details

We test whether contrast-set-aware configurations improve failure detection relative to the source-distribution baseline using seed-paired t -tests on full-curve functional CP ROC-AUC. Each comparison reports the mean difference relative to SAFE, with positive values indicating improvement over the baseline. We apply Bonferroni correction over $m = 12$ tests. Table 6 enumerates the p values.

D Additional Alpha Sweep Results

This section reports the full conformal-significance sweep used to support the α -marginalized results in the main paper. For each setting, we report both F1 and ROC-AUC. F1 measures calibrated rollout-level failure detection quality at each operating point, while ROC-AUC measures separability between successful and failed rollouts induced by the conformal threshold sweep.

D.1 Simulation Alpha Sweep Results

Tables 7 and 8 report simulation performance for π_0 and OpenVLA on LIBERO. The F1 table shows the practical detection quality at each risk tolerance, while the ROC-AUC table shows how the conformal threshold sweep separates successful and failed rollouts. Relative gains are computed with respect to SAFE at the same α .

Table 7: F1 vs. α for π_0 and OpenVLA in the LIBERO simulation suite, averaged over 30 seeds. The Relative gain $\Delta C_k = \text{F1}(C_k) - \text{F1}(\text{SAFE})$. C1=SAFE, C2=SAFECAST_{TRAINAUG}, C3=SAFECAST_{CALAUG}, and C4=SAFECAST.

α	π_0							OpenVLA						
	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$
0.02	0.880	0.861	0.836	0.944	-0.019	-0.044	+0.064	0.841	0.930	0.731	0.767	+0.089	-0.110	-0.074
0.05	0.845	0.890	0.841	0.914	+0.045	-0.004	+0.069	0.856	0.929	0.809	0.863	+0.072	-0.048	+0.007
0.10	0.807	0.883	0.880	0.895	+0.077	+0.074	+0.088	0.877	0.922	0.878	0.942	+0.045	+0.001	+0.065
0.15	0.786	0.859	0.861	0.871	+0.074	+0.075	+0.086	0.889	0.918	0.915	0.936	+0.028	+0.026	+0.047
0.20	0.769	0.837	0.828	0.846	+0.069	+0.060	+0.077	0.892	0.907	0.908	0.919	+0.015	+0.016	+0.027
0.25	0.755	0.809	0.796	0.820	+0.054	+0.041	+0.065	0.885	0.898	0.896	0.906	+0.013	+0.012	+0.021
0.30	0.736	0.790	0.772	0.797	+0.053	+0.036	+0.060	0.876	0.887	0.887	0.892	+0.011	+0.011	+0.016
0.35	0.729	0.774	0.752	0.778	+0.045	+0.023	+0.049	0.870	0.877	0.880	0.881	+0.007	+0.010	+0.011
0.40	0.724	0.762	0.737	0.760	+0.037	+0.013	+0.036	0.866	0.871	0.871	0.873	+0.005	+0.005	+0.007
0.45	0.715	0.752	0.722	0.747	+0.037	+0.008	+0.032	0.861	0.864	0.865	0.865	+0.004	+0.005	+0.004
0.50	0.704	0.741	0.715	0.733	+0.037	+0.011	+0.029	0.856	0.859	0.859	0.859	+0.003	+0.003	+0.002
0.60	0.689	0.718	0.697	0.712	+0.029	+0.008	+0.023	0.847	0.851	0.849	0.851	+0.003	+0.002	+0.004
0.70	0.676	0.696	0.683	0.695	+0.020	+0.006	+0.019	0.842	0.845	0.841	0.845	+0.004	-0.000	+0.003
0.80	0.662	0.678	0.670	0.679	+0.016	+0.008	+0.017	0.838	0.840	0.837	0.839	+0.002	-0.001	+0.001
0.90	0.652	0.660	0.653	0.658	+0.008	+0.001	+0.006	0.835	0.836	0.835	0.835	+0.001	-0.000	+0.000

D.2 Real-World DROID Alpha Sweep Results

Tables 9 and 10 report the real-world DROID alpha sweeps for π_0 and π_0 -FAST. The F1 table summarizes calibrated failure-detection quality at each deployment risk tolerance, while the ROC-AUC table summarizes the corresponding success/failure separability induced by the threshold sweep.

Table 8: Cumulative ROC-AUC vs. α for π_0 and OpenVLA in the LIBERO simulation suite, averaged over 30 seeds. The Relative Gain $\Delta C_k = \text{ROC}(C_k) - \text{ROC}(\text{SAFE})$. C1=SAFE, C2=SAFECAST_{TRAINAUG}, C3=SAFECAST_{CALAUG}, and C4=SAFECAST.

α	π_0							OpenVLA						
	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$
0.02	0.000	0.000	0.000	0.000	+0.000	+0.000	+0.000	0.000	0.000	0.000	0.000	+0.000	+0.000	+0.000
0.05	0.053	0.039	0.038	0.039	-0.014	-0.015	-0.014	0.036	0.046	0.010	0.019	+0.010	-0.025	-0.016
0.10	0.116	0.085	0.065	0.067	-0.031	-0.051	-0.050	0.076	0.154	0.042	0.083	+0.077	-0.034	+0.007
0.15	0.155	0.123	0.090	0.099	-0.033	-0.065	-0.056	0.137	0.242	0.094	0.178	+0.106	-0.043	+0.041
0.20	0.190	0.156	0.137	0.138	-0.034	-0.053	-0.052	0.201	0.322	0.160	0.313	+0.121	-0.041	+0.112
0.25	0.221	0.203	0.193	0.180	-0.017	-0.028	-0.041	0.267	0.387	0.243	0.393	+0.120	-0.024	+0.126
0.30	0.254	0.239	0.234	0.219	-0.015	-0.020	-0.035	0.341	0.463	0.313	0.483	+0.123	-0.027	+0.142
0.35	0.274	0.268	0.273	0.252	-0.006	-0.001	-0.022	0.380	0.526	0.365	0.551	+0.146	-0.015	+0.171
0.40	0.288	0.292	0.307	0.286	+0.004	+0.019	-0.002	0.412	0.567	0.424	0.604	+0.155	+0.012	+0.192
0.45	0.304	0.311	0.336	0.312	+0.008	+0.032	+0.009	0.453	0.609	0.460	0.666	+0.156	+0.008	+0.213
0.50	0.322	0.331	0.352	0.339	+0.009	+0.030	+0.017	0.487	0.647	0.503	0.709	+0.161	+0.016	+0.223
0.60	0.355	0.377	0.388	0.384	+0.022	+0.034	+0.029	0.542	0.707	0.586	0.761	+0.165	+0.045	+0.219
0.70	0.383	0.424	0.422	0.422	+0.041	+0.039	+0.039	0.595	0.742	0.644	0.812	+0.147	+0.049	+0.216
0.80	0.415	0.467	0.453	0.461	+0.052	+0.038	+0.046	0.631	0.781	0.693	0.853	+0.150	+0.062	+0.222
0.90	0.440	0.512	0.494	0.511	+0.071	+0.054	+0.071	0.670	0.807	0.725	0.883	+0.137	+0.055	+0.213

Table 9: F1 vs. α for π_0 and π_0 -FAST on DROID real-world rollouts, averaged over 30 seeds. $\Delta C_k = \text{F1}(C_k) - \text{F1}(\text{SAFE})$. C1=SAFE, C2=SAFECAST_{TRAINAUG}, C3=SAFECAST_{CALAUG}, and C4=SAFECAST.

α	π_0							π_0 -FAST						
	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$
0.02	0.148	0.388	0.108	0.203	+0.240	-0.040	+0.055	0.567	0.703	0.543	0.615	+0.137	-0.024	+0.049
0.05	0.153	0.413	0.128	0.244	+0.260	-0.025	+0.091	0.594	0.731	0.581	0.641	+0.136	-0.014	+0.046
0.10	0.169	0.453	0.178	0.303	+0.284	+0.008	+0.134	0.627	0.770	0.621	0.686	+0.143	-0.006	+0.059
0.15	0.206	0.483	0.242	0.338	+0.277	+0.036	+0.133	0.662	0.791	0.656	0.736	+0.129	-0.006	+0.074
0.20	0.256	0.504	0.287	0.426	+0.248	+0.031	+0.170	0.693	0.806	0.707	0.807	+0.113	+0.014	+0.113
0.25	0.300	0.538	0.329	0.467	+0.239	+0.029	+0.167	0.710	0.813	0.732	0.822	+0.103	+0.022	+0.111
0.30	0.331	0.556	0.357	0.485	+0.225	+0.026	+0.154	0.724	0.827	0.768	0.834	+0.103	+0.045	+0.110
0.35	0.361	0.576	0.393	0.500	+0.215	+0.031	+0.139	0.757	0.838	0.778	0.837	+0.081	+0.021	+0.080
0.40	0.372	0.583	0.421	0.525	+0.211	+0.049	+0.153	0.773	0.837	0.787	0.835	+0.064	+0.014	+0.062
0.45	0.412	0.595	0.442	0.550	+0.183	+0.030	+0.138	0.777	0.838	0.780	0.836	+0.061	+0.002	+0.059
0.50	0.431	0.608	0.465	0.581	+0.177	+0.034	+0.150	0.779	0.839	0.777	0.841	+0.060	-0.002	+0.062
0.60	0.459	0.617	0.522	0.604	+0.158	+0.062	+0.145	0.785	0.825	0.784	0.830	+0.039	-0.002	+0.044
0.70	0.514	0.632	0.567	0.632	+0.118	+0.052	+0.117	0.783	0.807	0.791	0.815	+0.025	+0.008	+0.033
0.80	0.553	0.646	0.602	0.650	+0.093	+0.049	+0.096	0.785	0.799	0.784	0.805	+0.015	-0.001	+0.020
0.90	0.591	0.654	0.635	0.665	+0.063	+0.045	+0.074	0.774	0.790	0.769	0.796	+0.016	-0.005	+0.022

E Additional Ablations

E.1 Training Exposure vs. Calibration Alignment

This ablation compares SAFECAST_{TRAINAUG} and SAFECAST_{CALAUG} to isolate the relative effects of contrast-set exposure during probe training and contrast-set alignment during conformal calibration. SAFECAST_{TRAINAUG} changes the learned failure-score function by training the probe on source and contrast-set rollouts, while SAFECAST_{CALAUG} keeps the probe trained on source rollouts but estimates conformal thresholds using augmented calibration trajectories. The results indicate that both mechanisms can improve robustness, but the dominant mechanism varies across policy backbones and deployment settings.

E.2 Additional Modality Analysis

The main paper reports F1 trends for visual-only, language-only, and joint visual-language contrast-set ablations. Table 11 summarizes the corresponding α -marginal cumulative ROC-AUC.

Table 10: Cumulative ROC-AUC vs. α for π_0 and π_0 -FAST on DROID real-world rollouts, averaged over 30 seeds. $\Delta C_k = \text{ROC}(C_k) - \text{ROC}(\text{SAFE})$. C1=SAFE, C2=SAFECAST_{TRAINAUG}, C3=SAFECAST_{CALAUG}, and C4=SAFECAST.

α	π_0							π_0 -FAST						
	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$	C1	C2	C3	C4	$\Delta C2$	$\Delta C3$	$\Delta C4$
0.02	0.000	0.000	0.000	0.000	+0.000	+0.000	+0.000	0.000	0.000	0.000	0.000	+0.000	+0.000	+0.000
0.05	0.001	0.023	0.001	0.003	+0.022	+0.000	+0.003	0.007	0.014	0.007	0.007	+0.007	+0.000	+0.000
0.10	0.002	0.058	0.004	0.014	+0.055	+0.002	+0.012	0.021	0.046	0.028	0.036	+0.025	+0.007	+0.015
0.15	0.008	0.089	0.015	0.031	+0.081	+0.007	+0.023	0.043	0.069	0.052	0.054	+0.026	+0.008	+0.011
0.20	0.015	0.114	0.024	0.066	+0.099	+0.009	+0.051	0.071	0.092	0.073	0.080	+0.021	+0.002	+0.009
0.25	0.027	0.147	0.041	0.096	+0.119	+0.013	+0.068	0.101	0.119	0.087	0.104	+0.018	-0.014	+0.003
0.30	0.045	0.170	0.059	0.114	+0.125	+0.015	+0.069	0.136	0.138	0.115	0.122	+0.001	-0.021	-0.015
0.35	0.062	0.197	0.084	0.141	+0.136	+0.022	+0.079	0.175	0.187	0.147	0.157	+0.013	-0.028	-0.017
0.40	0.072	0.209	0.107	0.174	+0.137	+0.034	+0.101	0.204	0.225	0.177	0.222	+0.021	-0.027	+0.018
0.45	0.092	0.235	0.124	0.199	+0.143	+0.032	+0.108	0.249	0.272	0.244	0.284	+0.023	-0.005	+0.035
0.50	0.103	0.249	0.137	0.230	+0.146	+0.035	+0.127	0.300	0.356	0.319	0.364	+0.057	+0.019	+0.065
0.60	0.132	0.262	0.190	0.273	+0.130	+0.058	+0.141	0.372	0.487	0.407	0.503	+0.115	+0.035	+0.131
0.70	0.167	0.290	0.245	0.325	+0.124	+0.078	+0.158	0.422	0.607	0.463	0.605	+0.185	+0.041	+0.183
0.80	0.225	0.308	0.288	0.368	+0.083	+0.063	+0.143	0.470	0.655	0.544	0.659	+0.185	+0.074	+0.189
0.90	0.265	0.322	0.345	0.394	+0.057	+0.081	+0.129	0.572	0.709	0.682	0.701	+0.136	+0.110	+0.129

Table 11: α -marginal cumulative ROC-AUC for contrast-set modality ablations.

Contrast sets	OpenVLA ROC-AUC $\uparrow$	π_0 ROC-AUC $\uparrow$
Visual only	0.484	0.469
Language only	0.484	0.441
Visual + Language (SAFECAST)	0.561	0.282

E.3 Additional Sim-to-Real Analysis

The main paper evaluates a simulation-trained probe calibrated on real-world contrast-set trajectories. Table 12 compares α -marginal F1 and cumulative ROC-AUC under three probe/calibration sources. The sim-only row reports in-simulation SAFECAST performance; the transfer row uses a simulation-trained π_0 probe with real contrast-set calibration and evaluates on real DROID rollouts.

Table 12: α -marginal failure detection on real DROID rollouts under different probe/calibration sources.

Configuration	F1 $\uparrow$	ROC-AUC $\uparrow$
Real-only probe and calibration	0.478	0.228
Sim-trained probe + real CS calibration	0.523	0.410

E.4 Low- and High-Alpha Regimes

The full α sweeps in Appendix D summarize performance across deployment risk tolerances. Here, we isolate low- and high- α regimes to show how methods behave under permissive versus conservative intervention settings. In our functional CP setup, higher α values correspond to lower risk thresholds and therefore more conservative intervention, while lower α values are more permissive and reduce unnecessary alarms.

E.4.1 Simulation Results

Figures 6 and 7 show simulation performance in the low- and high- α regimes. The high- α regime is most relevant for safety-critical deployment because it emphasizes conservative failure detection under shifted conditions. The low- α regime provides a complementary view of whether methods remain stable when fewer interventions are preferred.

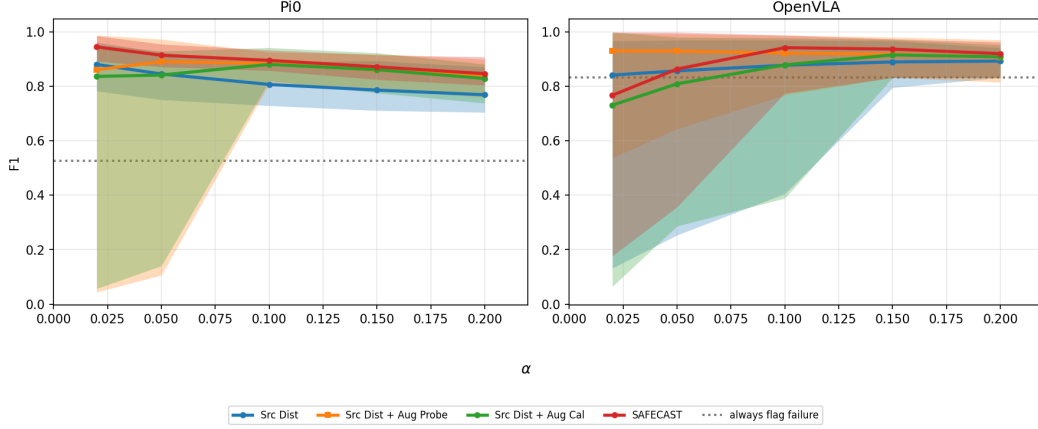


Figure 6: Simulation failure detection in the low- α regime, where thresholds are more permissive and fewer rollouts are flagged.

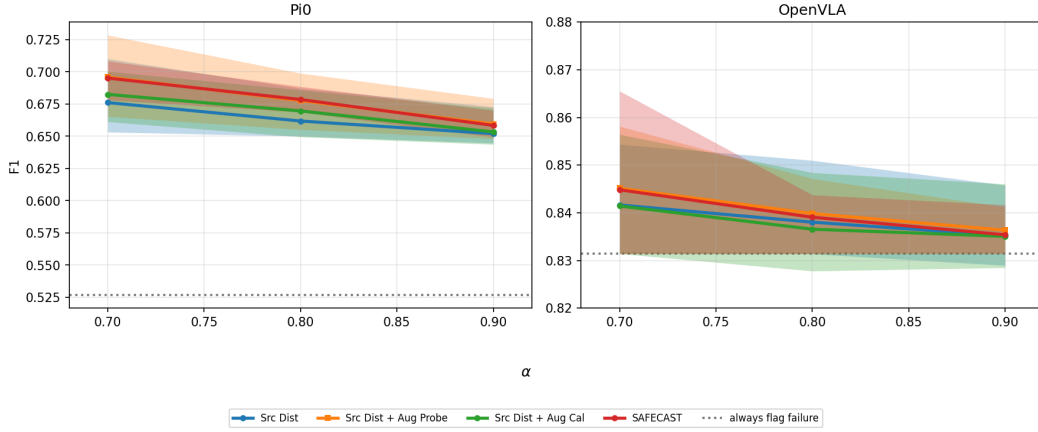


Figure 7: Simulation failure detection in the high- α regime, where thresholds are more conservative and failures are flagged more aggressively.

E.4.2 Real-World Results

Figures 8 and 9 show the corresponding real-world DROID trends. High- α performance is especially important in real deployment because missed failures can lead to unsafe or unrecoverable robot behavior. Together, the low- and high- α plots show whether gains persist across both permissive and conservative operating regimes.

F Additional Contrast Set Variations and Per-Shift Results

We developed and analyzed the effects of an extended set of vision, language, and anti-goal perturbations to augment the single distractor object (vision) and command paraphrase (language) perturbations explored in our primary experiments. We find that naively adding perturbations to the SAFECAST training and calibration set does not assist with failure detection at inference time. Pooling all proposed visual, language, or both visual and language perturbations generally underperforms using only the distractor object and language paraphrase perturbations for training and calibrating the SAFECAST probe for both π_0 and OpenVLA on both F1 and ROC-AUC metrics of failure detection. Additionally, we find that multiple family-level variations outperform individual variations for lighting, texture, and paraphrase families. Finally, we explore anti-goal perturbations where a model is instructed *not* to do the original task, and find that these do not improve failure

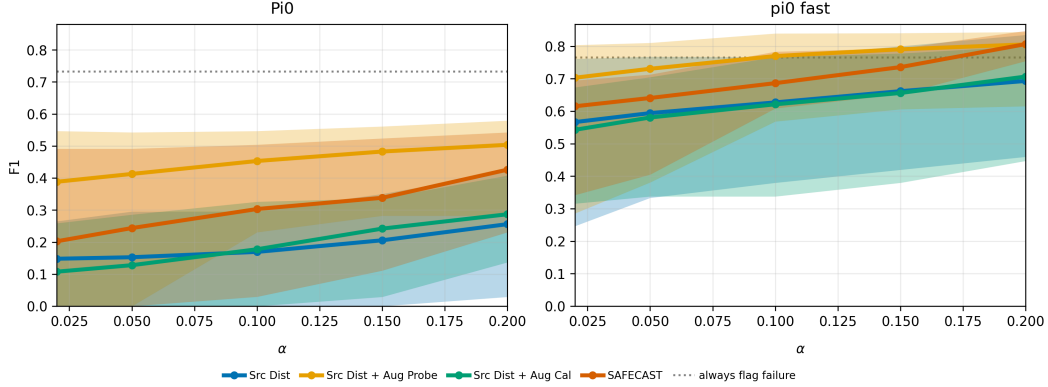


Figure 8: Real-world failure detection in the low- α regime, emphasizing permissive operation with fewer interventions.

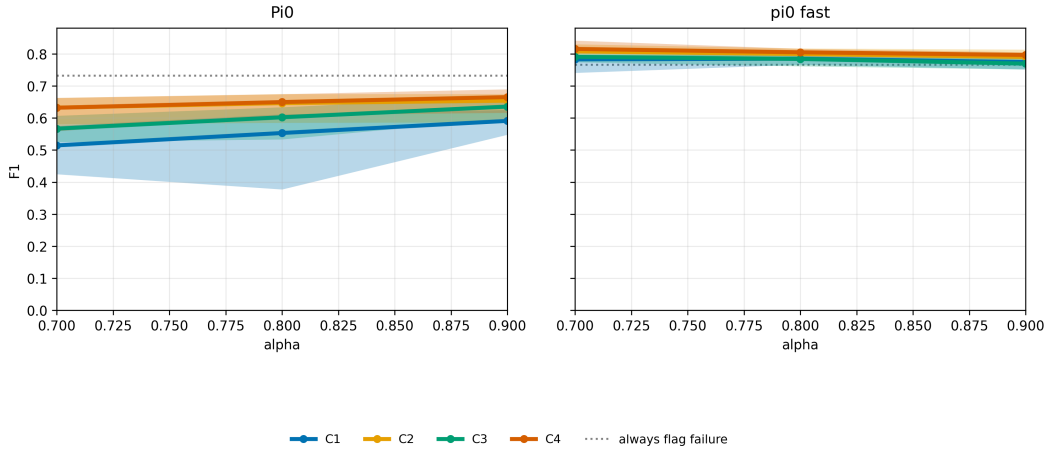


Figure 9: Real-world failure detection in the high- α regime, emphasizing conservative intervention when missed failures are costly.

detection. However, these anti-goal variations applied at inference time can reduce the π_0 success rate from the 90% range to the 50% range. The anti-goal variation may prove a useful sanity check for developing VLA benchmarking and checking for overfitting in general; we will explore this line of thought in upcoming research.

F.1 Perturbation Family Definitions

We document the additional contrast variations used to construct extended LIBERO contrast-set pools for SAFECAST ablations, together with per-family F1/ROC-AUC and precision-recall curves for π_0 and OpenVLA. We organize these variations into three families: vision variations, language variations, and anti-goal variations. Vision and language variations are goal-preserving contrast sets: they modify the observation or instruction while keeping the original BDDL goal unchanged. Anti-goal variations are treated as a separate family; these modify the language instruction to negate the original goal, reflected in a change to the sign of the BDDL goal evaluation. All rollouts are collected from frozen VLAs under perturbed conditions, filtered with DTW-based active rejection when used for contrast-set training, and evaluated on LIBERO-Plus test rollouts.

The extended contrast variations are grouped as follows:

- **Vision variations** preserve the original instruction and BDDL success predicate. This family includes lighting changes and tabletop texture changes.

LIBERO-Spatial visual contrast-set variations

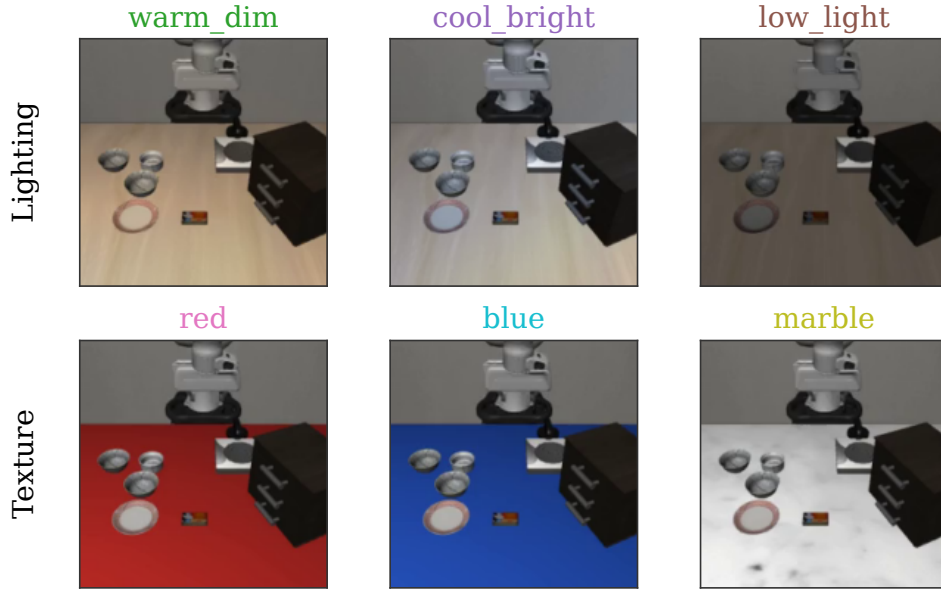


Figure 10: Examples of goal-preserving visual contrast variations used in the extended LIBERO contrast-set pool. Each panel shows the first frame of the same task family under a different visual condition, including lighting and tabletop texture changes. These perturbations preserve the instruction, object layout, and BDDL success predicate, so any performance change reflects sensitivity to visual appearance rather than a change in task semantics.

- **Language variations** are instruction-level shifts that preserve the original task semantics and BDDL success predicate. This family includes passive-voice instructions and negation-except paraphrases.
- **Anti-goal variations:** goal-reversal tasks that intentionally change the task objective and BDDL success predicate. These are evaluated separately and are not pooled with goal-preserving language variations.

We next define each of these families and the variations they comprise.

F.2 Lighting Variations

We apply three MuJoCo lighting configurations to the LIBERO-Spatial tabletop scene (Figure 10). Each configuration modifies diffuse (specular) color, light position, direction, and, for the low light configuration, skybox ambient color. Table 13 summarizes the settings; rollouts use the same task instructions and success predicates as the source distribution.

F.3 Texture Variations

We replace the default tabletop appearance with three surface styles. Object geometry, placement, and task semantics are unchanged. Table 14 lists the variations and representative RGB/texture settings.

Example. For the task *pick up the black bowl on the stove and place it on the plate*, the only change under the red variation is that the robot observes a red tabletop instead of the default wood-like surface; the instruction and goal remain identical.

Table 13: Lighting contrast-set variations for LIBERO-Spatial. Each lighting condition changes scene illumination while preserving the robot task, object layout, instruction, and BDDL success predicate. These shifts test whether SAFECAST remains robust when the same task is observed under different photometric conditions, such as dim, bright, or color-shifted lighting.

Variation	Description
warm_dim	Warm, dimmer front-right lighting with amber diffuse tones and reduced specular highlights.
cool_bright	Cool blue-tinted, brighter overhead lighting with higher diffuse/specular intensity.
low_light	Very low scene illumination with dim skybox ambient light, producing near-dark conditions.
combined_lighting	Pooled training/calibration rollouts from all three single lighting shifts.

Table 14: Table-surface texture contrast-set variations for LIBERO-Spatial. Each texture condition changes the visual appearance of the tabletop while leaving object geometry, object placement, instruction, and task success predicate unchanged. These variations test whether the failure detector relies on brittle background or surface appearance cues rather than task-relevant behavior.

Variation	Mode	Description / example
red	Flat color	Saturated red tabletop, RGB (0.78, 0.18, 0.16).
blue	Flat color	Saturated blue tabletop, RGB (0.16, 0.34, 0.78).
marble	Image texture	White marble floor texture mapped to the table surface.
combined_texture	Pooled	Pooled rollouts from red, blue, and marble.

F.4 Language Variations

Beyond the LLM-generated semantic paraphrases in Appendix A.2, we evaluate two additional goal-preserving language shifts on LIBERO-Spatial: passive voice and neg_except. Both perturbations change the surface form or syntactic structure of the instruction while preserving the target object, action, goal location, and BDDL success predicate. Table 15 contrasts these perturbations on a representative task.

Table 15: Goal-preserving language contrast variations with representative instructions. Passive voice and neg_except change the syntactic form of the command while preserving the target object, action, goal location, and BDDL success predicate. These examples test whether the policy and failure detector remain robust to linguistic reformulations that should imply the same robot trajectory.

Variation	Example instruction	Success criterion
Source (active)	Pick up the black bowl on the stove and place it on the plate.	Bowl on plate
passive	The black bowl on the stove is picked up and placed on the plate.	Bowl on plate; unchanged
neg_except	Pick up the black bowl on the stove and do not place it anywhere except on the plate.	Bowl on plate; unchanged

F.4.1 Passive-voice paraphrases

Passive instructions are generated by converting each stock LIBERO-Spatial BDDL instruction from active to passive voice while preserving the target object, action, and goal location. We use one passive variant per task. Given an active instruction of the form “Pick up [source description] and place it on the plate,” the passive form is

“The [source description] is picked up and placed on the plate.”

No new objects, goals, or spatial references are introduced.

F.4.2 Negation-except paraphrases

`neg_except` applies a goal-preserving negation template that forbids incorrect destinations but still requires placing the bowl on the plate. The template is:

“Pick up [source description] and do not place it anywhere except on the plate.”

The BDDL success predicate remains unchanged, e.g., `(On akita_black_bowl_1 plate_1)`. This perturbation tests whether the policy can parse negated phrasing without changing the intended task.

F.5 Anti-Goal Variations

Anti-goal variations are not language-only perturbations. They define a separate contrast family in which the task objective itself is reversed along with the language instruction. This perturbation is the only goal-altering variation considered in our study so far. Unlike passive voice or `neg_except`, which preserve the original task semantics, anti-goal variations change the BDDL success predicate and therefore correspond to a different goal condition.

For each LIBERO-Spatial task, we generate a companion BDDL file using `scripts/libero_anti_goal.py`. The anti-goal construction modifies three elements:

1. **Language:** the instruction is rewritten to explicitly avoid the original target location.
2. **Goal predicate:** the success predicate is changed from the original goal to its negation, e.g., from `(On bowl plate)` to `(Not (On bowl plate))`.
3. **Initial state:** when needed, the target object is initialized so that satisfying the anti-goal requires behavior different from the source task.

Table 16: Anti-goal contrast variation. Unlike goal-preserving language perturbations, anti-goal changes the task objective and therefore changes the BDDL success predicate. This setting tests whether the failure detector behaves differently under explicit goal reversal, where the instruction no longer describes the original source task.

Setting	Example instruction	Success criterion
Source task	Pick up the black bowl on the stove and place it on the plate.	Bowl on plate
Anti-goal task	Pick up the black bowl and place it anywhere except on the plate.	Bowl not on plate

Because anti-goal changes the success predicate, it is evaluated as a separate family and is not included in `combined_language`. This separation avoids conflating semantics-preserving language robustness with robustness to explicit goal reversal.

F.6 Active Rejection During Contrast-Set Collection

All contrast-set rollouts in this section, lighting, texture, passive, `neg_except`, are collected with DTW-based active rejection when used for probe training. During collection, each candidate rollout is represented by a translation-normalized end-effector trajectory; FastDTW distance is normalized by warping-path length and compared against already-accepted rollouts in the same task-and-outcome pool. Candidates with normalized DTW distance below $\tau_{DTW} = 0.02$ are rejected. This reduces near-duplicate trajectories in $\mathcal{D}_{\phi, CS}^{train}$ and increases diversity among accepted perturbation rollouts before probe training and conformal calibration.

F.7 SAFECAST Pooling Configurations

Individual perturbations above are pooled into broader SAFECAST training/calibration subsets:

- **Pooled lighting** (`c4_combined_lighting`): `warm_dim + cool_bright + low_light`.

- **Pooled texture** (`c4_combined_texture`): red + blue + marble.
- **Vision** (`c4_combined_vision`): lighting + texture.
- **Language** (`c4_combined_language`): passive + `neg_except`.
- **Vision + language** (`c4_combined_vision_language`): vision variations plus passive and `neg_except`.
- **Anti-goal** (`c4_anti_goal`): goal-reversal tasks only.

Each SAFECAST ablation trains and calibrates the probe on source rollouts plus the listed contrast pool, then evaluates on LIBERO-Plus unseen rollouts. When additional perturbation pools are added, the available training/calibration pool becomes larger; however, batch size and number of training iterations are kept fixed so that each probe receives the same number of gradient updates. Anti-goal is always kept separate from the language pool because it changes the task success predicate.

F.8 F1 and Cumulative ROC-AUC Curves

Figures 11–15 report seed-averaged F1 and cumulative ROC-AUC versus conformal risk level α for π_0 and OpenVLA. Each figure uses a 2×2 layout: F1 is shown in the top row, cumulative ROC-AUC in the bottom row, π_0 in the left column, and OpenVLA in the right column. Shaded bands denote p10–p90 across 30 random seeds. F1 axes are fixed to $[0.5, 1.0]$ for readability. Curves labeled C1 and C4 provide source-only and full contrast-set references from the main paper.

How to read the families.

- **Combined** (Fig. 11): compares vision, language, and joint vision+language SAFECAST configurations.
- **Lighting** (Fig. 12): compares single lighting shifts, `combined_lighting`, and the `combined_vision` reference.
- **Texture** (Fig. 13): compares single texture shifts, `combined_texture`, and the `combined_vision` reference.
- **Language** (Fig. 14): compares passive, `neg_except`, and `combined_language`.
- **Anti-goal** (Fig. 15): evaluates goal-reversal contrast only and tests failure detection under instruction–goal mismatch.

F.9 LIBERO-Plus Evaluation Class Balance

Table 17 reports the base policy success rates on the LIBERO-Plus test rollouts used for SAFECAST evaluation. These success rates determine the success/failure class balance for failure detection: π_0 succeeds on a majority of LIBERO-Plus rollouts, while OpenVLA fails on a majority of rollouts. This difference is important when interpreting ROC-AUC and precision–recall curves, because the two policies induce different failure prevalences on the same shifted evaluation suite.

Table 17: Policy success rate (SR) on the LIBERO-Plus test set used for SAFECAST evaluation.

Policy	Successes	Rollouts	SR
π_0	1,546	2,406	64.3%
OpenVLA	693	2,406	28.8%

Because failure detection treats failed rollouts as the positive class, the implied failure rates are 35.7% for π_0 and 71.1% for OpenVLA. This difference partly motivates the precision–recall diagnostics in Appendix F.10, since PR curves are more sensitive than ROC curves to the positive-class prevalence.

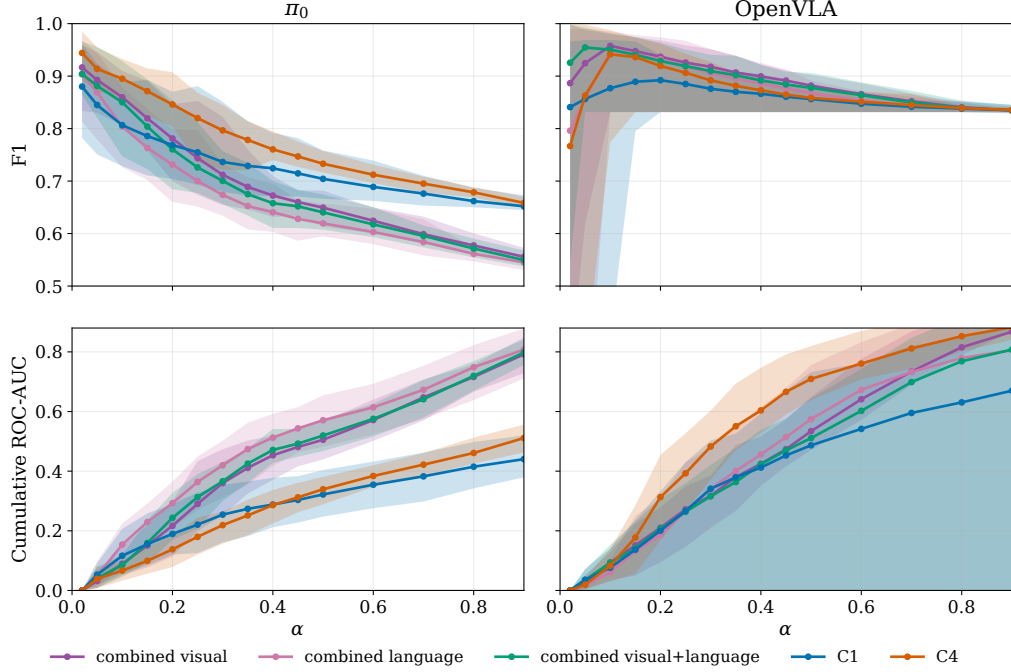


Figure 11: Combined contrast-family ablations for SAFECAST on LIBERO-Plus. The top row reports F1 versus conformal risk level α , where higher curves indicate better calibrated failure detection at that operating point. The bottom row reports cumulative ROC-AUC versus α , where higher curves indicate stronger separation between successful and failed rollouts across the conformal threshold sweep. The left column shows π_0 and the right column shows OpenVLA. For π_0 , the original C4 reference maintains the strongest F1 across most of the sweep, while the combined vision-language pool improves ROC-AUC at higher α , suggesting better separability but a precision–recall trade-off. For OpenVLA, C4 and the combined vision-language setting are strongest overall, indicating that OpenVLA benefits most when visual and language shifts are modeled jointly. Overall, joint vision-language contrast sets provide the most consistent robustness signal, but their benefit is clearer in ROC-AUC than in F1 for π_0 .

F.10 Precision–Recall Curves

We additionally report precision–recall diagnostics for the extended contrast-set perturbations. These curves are included because some perturbation families, especially for π_0 , produce unusual ROC-AUC trends that may be affected by class imbalance or threshold behavior. Precision–recall curves provide a complementary view of whether increased failure recall is accompanied by useful precision.

For each conformal risk level α in the functional CP sweep, we extract F1 and true-positive rate (TPR) on LIBERO-Plus test rollouts and derive precision as

$$\text{Prec}(\alpha) = \frac{\text{F1}(\alpha) \cdot \text{TPR}(\alpha)}{2 \text{TPR}(\alpha) - \text{F1}(\alpha)}.$$

Here, TPR is equivalent to recall for the failure-detection task. When $2\text{TPR}(\alpha) - \text{F1}(\alpha) = 0$, precision is undefined and the corresponding point is omitted. Points are connected in α order. Thus, these curves summarize how calibrated failure detection trades precision and recall as the conformal threshold varies.

Figures 16–20 report CP- α precision–recall curves for the combined, lighting, texture, language, and anti-goal perturbation families.

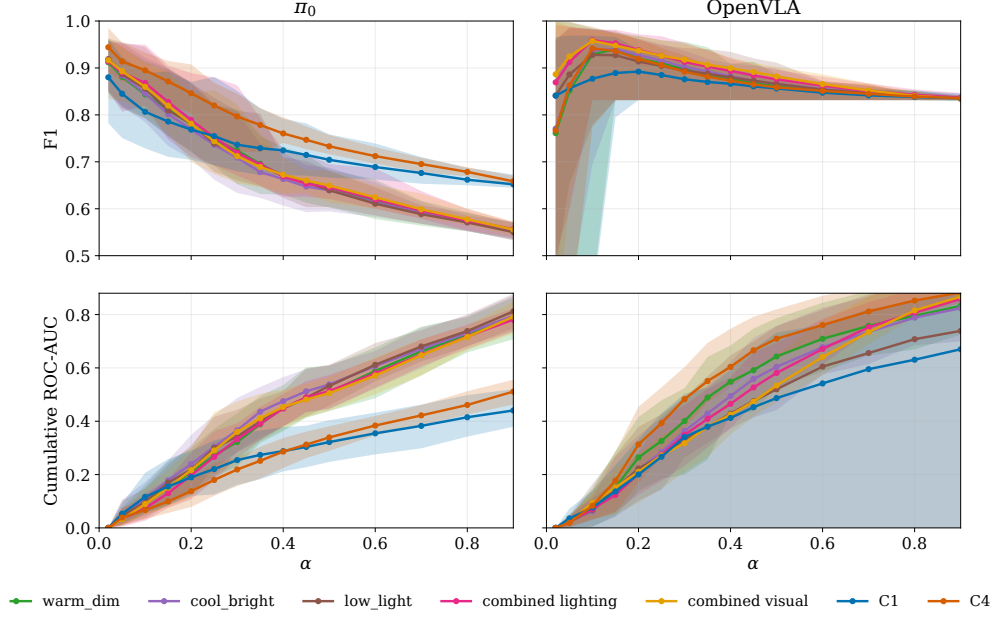


Figure 12: Lighting variation ablations for SAFECAST on LIBERO-Plus. The curves compare individual lighting shifts, pooled lighting, the combined-vision pool, and the C1/C4 references. Higher F1 in the top row indicates better calibrated failure detection at a fixed conformal risk level α , while higher cumulative ROC-AUC in the bottom row indicates better success/failure separability as the conformal threshold sweep expands. For π_0 , lighting-only variants improve separability over C1 at higher α , but they do not consistently exceed the original C4 reference in F1, suggesting that photometric diversity alone is not sufficient for the best calibrated detector. For OpenVLA, the C4 and combined-vision curves remain among the strongest, indicating that lighting changes are useful but work best when combined with other visual perturbations. Thus, lighting perturbations improve coverage of visual shift, but pooled visual diversity is generally more reliable than any single lighting condition.

The precision–recall diagnostics are especially important for interpreting the π_0 results. Because π_0 succeeds on many source-distribution and mildly shifted rollouts, some perturbation settings can produce class imbalance in the evaluation labels. In such cases, ROC-AUC may improve even when calibrated failure detection does not uniformly improve across operating points. The CP- α precision–recall curves decompose this behavior by showing whether a perturbation mainly increases recall, preserves precision, or introduces a precision–recall trade-off.

F.11 Discussion and Key Takeaways

We developed and analyzed the effects of an extended set of vision, language, and anti-goal perturbations to augment the single distractor object (vision) and command paraphrase (language) perturbations explored in our primary experiments.

Naively Adding Perturbations Does Not Help Failure Detection. We find that pooling all proposed visual, language, or both visual and language perturbations generally underperforms using only the distractor object and language paraphrase perturbations for training and calibrating the SAFECAST probe for both π_0 and OpenVLA on both F1 and ROC-AUC metrics of failure detection (Figure 11). A notable exception is the cumulative ROC-AUC of the π_0 model, which is consistently improved by these expanded contrast sets across sensitivity values α . A partial accounting of this effect comes from the class imbalance in π_0 failure detection on the LIBERO-Plus test set we utilize; π_0 achieves success on most test instances, making sensitivity to correctly identifying failure more visibly rewarded in ROC-AUC than F1, which is more easily flattened by majority class predictions.

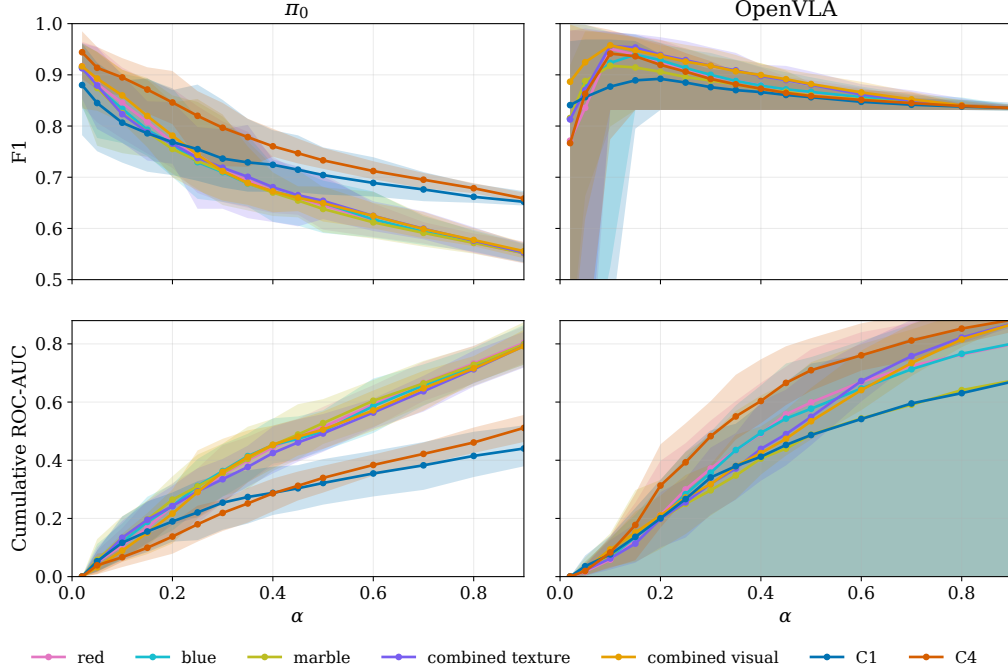


Figure 13: Texture variation ablations for SAFECAST on LIBERO-Plus. The curves compare red, blue, and marble tabletop textures, pooled texture, the combined-vision pool, and the C1/C4 references. Because these perturbations preserve the instruction, object layout, and success predicate, performance differences measure sensitivity to background and surface appearance rather than changes in task semantics. For π_0 , texture variants can improve cumulative ROC-AUC relative to C1 at larger α , showing that surface-appearance diversity helps separate successful and failed rollouts, but F1 remains strongest or most stable for the C4 reference. For OpenVLA, C4 and combined-vision remain the most reliable overall, with texture-only shifts providing smaller gains. This suggests that texture changes are useful as part of a broader visual contrast pool, but are weaker than the full visual contrast setting by themselves.

See the precision and recall curves presented in Figures 16 through 20 for π_0 and OpenVLA for a visual accounting of this phenomena; in particular, π_0 SAFE and SAFECAST probes trained across nearly every evaluated contrast set perturbation result in 1.0 recall regardless of α sensitivity.

Multiple Family-level Variations Outperform Individual Variations. In the cases of lighting variations (Figure 12), tabletop texture variations (Figure 13), and passive voice and negation-except paraphrases (Figure 14), we find that combining these perturbations outperforms or matches the performance of each lighting/texture/paraphrase method individually in F1 and ROC-AUC on failure detection. Similarly to the findings of naive perturbation combinations, the primary SAFECAST method utilizing only distractor objects and language instruction paraphrases outperforms these combined perturbations in most cases, with π_0 ROC-AUC again providing a counter-trend that we attribute partially to class imbalance.

Anti-goal Perturbations Do Not Improve Failure Detection but they are really, really funny. Figure 15 reveals the same trends for anti-goal as other additional tested vision and language paraphrases: it is outperformed by vanilla SAFECAST except for π_0 ROC-AUC, where class imbalance lets anti-goal shine by achieving a better precision-recall tradeoff (Figure 20). However, a more interesting finding and takeaway for later work is that anti-goal variations applied at inference time to generate probe calibration data reduced the π_0 success rate from the 90% range to the 50% range, effectively rendering its success a coin toss as to whether the system would ignore the command it was told to ignore, or skip over the negation term and fall into an overfit trajectory execution any-

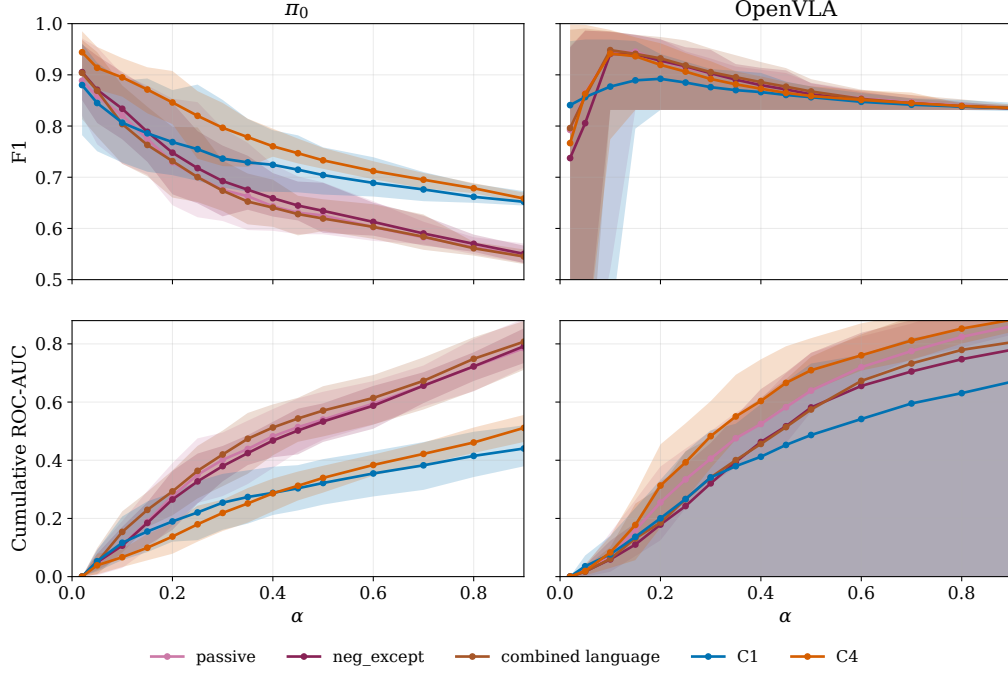


Figure 14: Goal-preserving language variation ablations for SAFECAST on LIBERO-Plus. The curves compare passive voice, `neg_except`, pooled language, and the C1/C4 references. All language variants preserve the original BDDL success predicate, so changes in F1 and ROC-AUC reflect robustness to instruction form rather than a change in the desired goal. For π_0 , passive and `neg_except` improve ROC-AUC over C1 at higher α , indicating that syntactic language diversity can improve failure separability, but their F1 curves are lower than or comparable to the C4 reference across much of the sweep. For OpenVLA, language variants closely track the stronger C4 behavior, suggesting that OpenVLA is especially sensitive to, and benefits from, language-side contrast information. Overall, goal-preserving language perturbations are useful, but the strongest robustness comes from combining language with visual contrast sets rather than using language alone.

way. The anti-goal variation may prove a useful sanity check for developing VLA benchmarking and checking for overfitting in general; we will explore this line of thought in upcoming research.

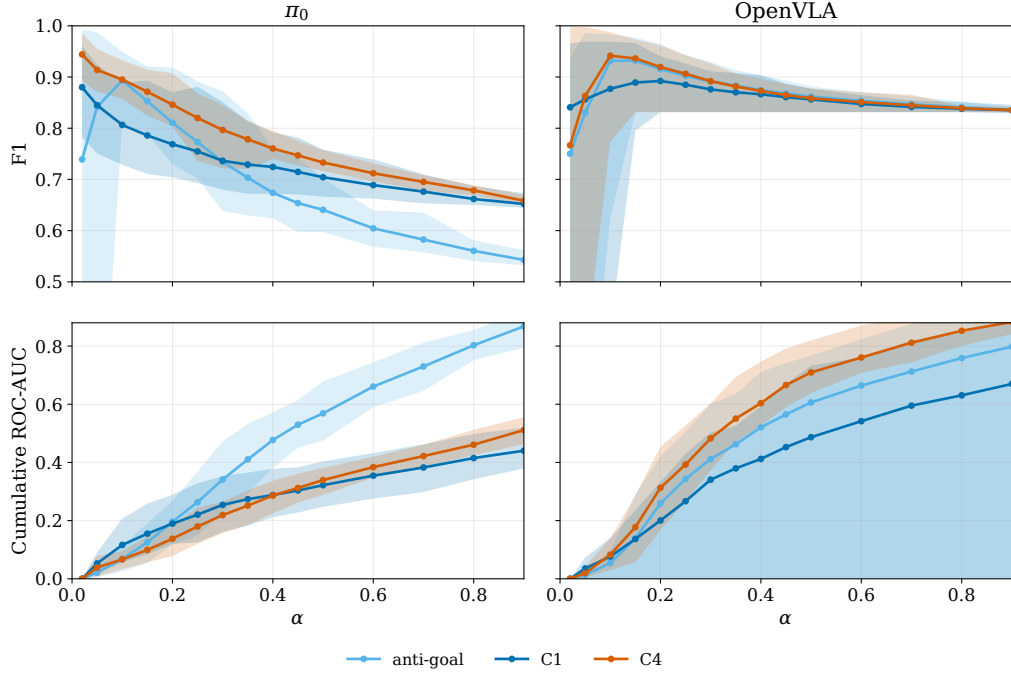


Figure 15: Anti-goal ablation for SAFECAST on LIBERO-Plus. Anti-goal is evaluated separately from language perturbations because it reverses the task objective and changes the BDDL success predicate. The top row shows F1 across α , and the bottom row shows cumulative ROC-AUC; higher curves are better for both metrics. For π_0 , anti-goal produces high ROC-AUC at larger α but weaker F1 than C4, indicating that goal reversal can make successes and failures more separable while also changing the calibrated precision–recall trade-off. For OpenVLA, C4 remains stronger overall, especially in ROC-AUC, suggesting that ordinary contrast-set training and calibration are more useful than anti-goal exposure for deployment-style LIBERO-Plus shifts. This result supports treating anti-goal as a distinct stress test rather than pooling it with goal-preserving language perturbations.

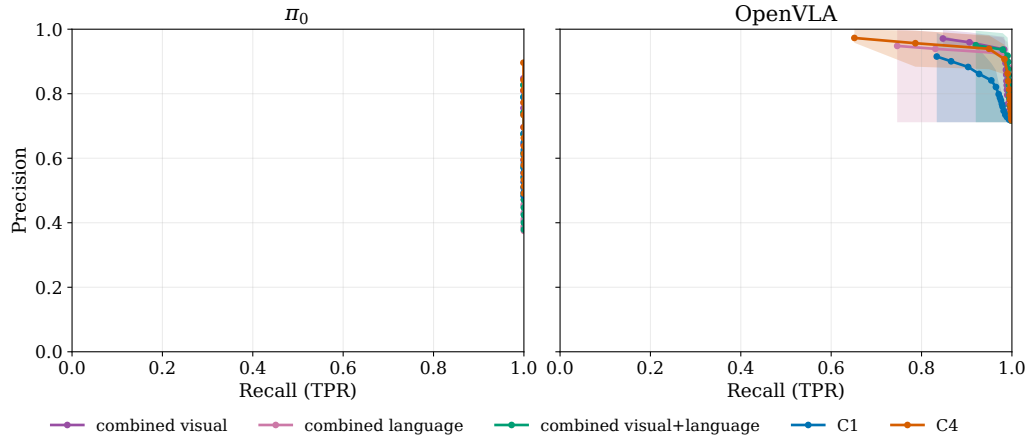


Figure 16: Precision–recall curves for combined SAFECAST ablations on LIBERO-Plus. Each point corresponds to one conformal risk level α ; curves farther toward the upper-right are better because they achieve higher precision at higher recall. The left panel shows π_0 and the right panel shows OpenVLA. For π_0 , most curves operate near very high recall, so differences are mainly visible in precision; this indicates that several settings flag most failures, but differ in how many false alarms they introduce. For OpenVLA, the C4 and joint vision-language curves lie closest to the upper-right envelope, showing the best balance between catching failures and avoiding unnecessary interventions. This PR view clarifies that ROC-AUC gains should be interpreted together with precision, especially for π_0 where class balance makes many methods appear recall-heavy.

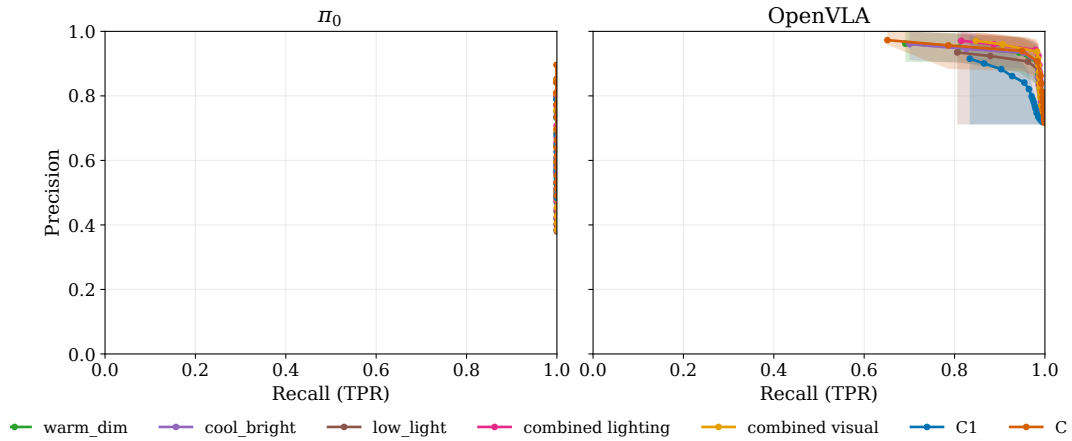


Figure 17: precision–recall curves for lighting variation ablations on LIBERO-Plus. Curves closer to the upper-right indicate better calibrated failure detection, with higher recall meaning more failures are caught and higher precision meaning fewer false alarms. For π_0 , lighting variants largely cluster at high recall, so the key distinction is whether they preserve precision while flagging many failures. For OpenVLA, C4 and the stronger pooled visual settings maintain the best precision–recall trade-off, while individual lighting shifts are more variable. This suggests that lighting perturbations help expose photometric failure modes, but lighting alone is less reliable than broader visual contrast diversity.

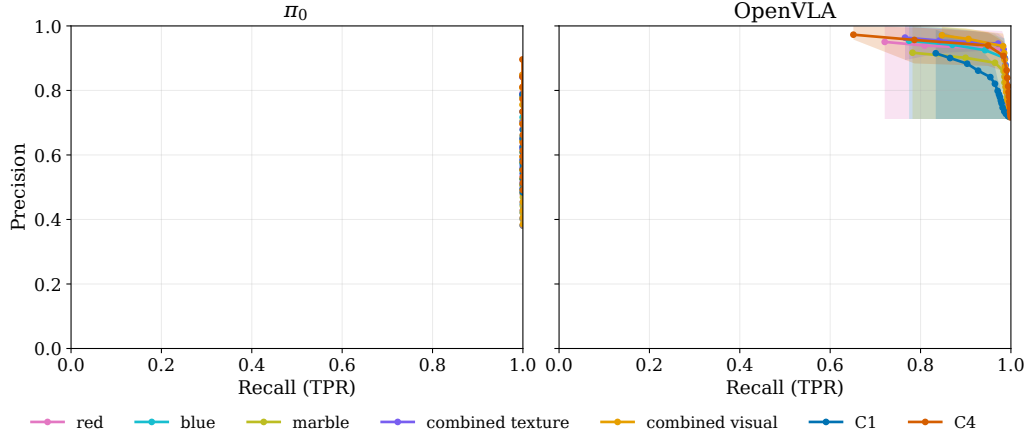


Figure 18: Precision–recall curves for texture variation ablations on LIBERO-Plus. The curves compare individual tabletop texture shifts, pooled texture, combined vision, and the C1/C4 references. Higher and farther-right curves are better because they detect more failed rollouts while maintaining higher precision. For π_0 , texture variants again concentrate near high recall, indicating aggressive failure coverage, but precision is the deciding factor for practical usefulness. For OpenVLA, the C4 and combined-vision references remain closest to the upper-right region, suggesting that texture changes are helpful but work best as part of a larger visual perturbation pool. These curves support the interpretation from the F1/ROC-AUC plots: texture diversity improves visual robustness, but by itself is not consistently the best contrast-set family.

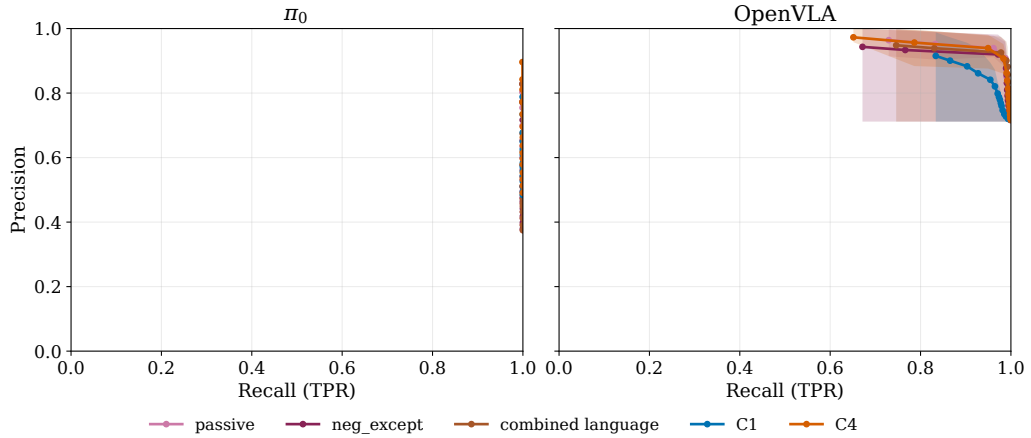


Figure 19: Precision–recall curves for goal-preserving language variation ablations on LIBERO-Plus. The curves compare passive voice, `neg_except`, pooled language, and the C1/C4 references. Because all variants preserve the original task goal, upper-right movement in this plot indicates improved robustness to instruction phrasing rather than to a changed task objective. For π_0 , the language variants primarily affect precision at very high recall, showing that they tend to detect many failures but may differ in false-alarm behavior. For OpenVLA, language contrast sets are competitive with C4, indicating that language-side perturbations provide a strong robustness signal for this policy. Overall, these PR curves show that passive and negation-based instructions help evaluate whether failure detection remains useful under syntactic instruction shifts..

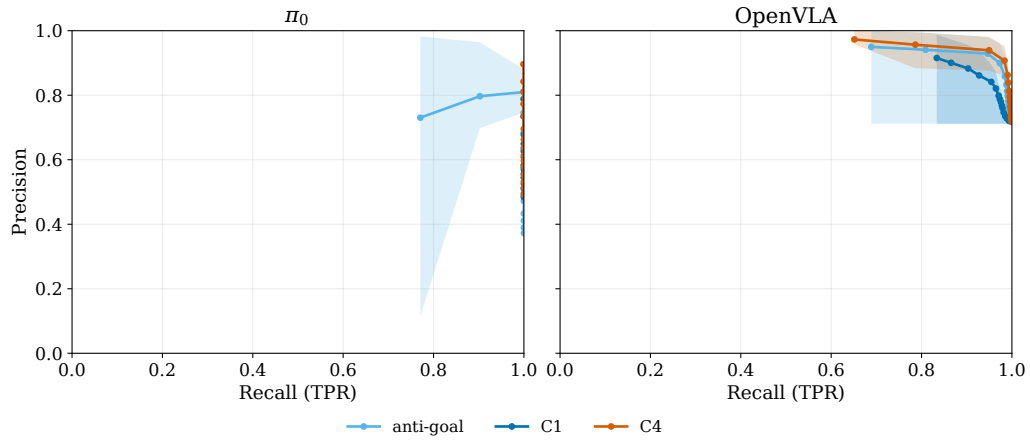


Figure 20: Precision–recall curves for anti-goal contrasts on LIBERO-Plus. Anti-goal reverses the task objective and is therefore shown separately from goal-preserving language perturbations. Curves closer to the upper-right indicate better practical failure detection, combining high failure recall with high precision. For π_0 , the anti-goal curve reaches high recall but has more variable precision, consistent with the F1/ROC-AUC result that anti-goal improves separability without necessarily improving calibrated detection quality. For OpenVLA, C4 maintains a stronger precision–recall trade-off than anti-goal, suggesting that goal-reversal data is less aligned with the LIBERO-Plus deployment shifts than the original contrast-set construction. This confirms that anti-goal is best interpreted as a severe stress test rather than a replacement for goal-preserving language contrast sets.